

Государственное бюджетное профессиональное образовательное учреждение
«Академия Инженерных Технологий и Управления»

МЕТОДИЧЕСКОЕ ПОСОБИЕ

по выполнению практического задания

Модули 1-2.

**Участие в проектировании архитектуры
интеллектуальных интегрированных систем**

Специальность: 09.02.08

Интеллектуальные Интегрированные Системы

Форма обучения: очная

Составлено:

Преподаватель ПМ.03

_____/_____
(подпись) (Ф.И.О.)

Утверждено:

Председатель ПЦК специальности

_____/_____
(подпись) (Ф.И.О.)

2026

АННОТАЦИЯ

Настоящее методическое пособие предназначено для студентов, выполняющих практическое задание по Модулям 1 и 2 «Участие в проектировании архитектуры интеллектуальных интегрированных систем» в рамках специальности 09.02.08 «Интеллектуальные Интегрированные Системы».

Модуль 1 охватывает базовое понимание визуального редактора карты на Python и PyQt6. Модуль 2 расширяет приложение: добавляется журналирование событий в базу данных SQLite, управление физическим светофором через интерфейс Arduino по последовательному порту, а также автоматический и ручной режимы работы светофора непосредственно из интерфейса программы.

Пособие содержит теоретические сведения, необходимые для понимания принципов построения графических приложений на языке Python с применением библиотеки PyQt6, а также описание архитектурных решений, использованных при разработке проекта «Интеллектуальная Дорожная Система».

Целевое назначение данного документа — сформировать у студента глубокое понимание задания модуля. Пособие не является пошаговой инструкцией с готовыми ответами: оно объясняет концептуальную основу каждого решения, что позволяет студенту самостоятельно реализовать задание и успешно представить решение.

Параметр	Значение
Дисциплина	МДК 03. Участие в разработке приложений взаимодействия с интеллектуальными интегрированными системами
Вид занятия	Практическое задание (демонстрационный экзамен)
Программное обеспечение	Python 3.10+, PyQt6
Форма отчётности	Работающая программа
Год составления	2026

СОДЕРЖАНИЕ

Аннотация.....	2
Содержание.....	3
ВВЕДЕНИЕ.....	7
ГЛАВА 1. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYQT6.....	10
1.1. Введение к главе	10
1.2. Подготовка рабочей среды	11
1.2.1. Установка зависимостей	11
1.2.2. Файл запуска run.bat и принцип рабочего каталога.....	12
1.3. Архитектура приложения	12
1.3.1. Три основных класса.....	12
1.3.2. Двухслойная модель данных	13
1.3.3. Порядок инициализации и взаимные ссылки.....	13
1.4. Шаг 1 — Константы и вспомогательные функции.....	13
1.4.1. Импорт и параметры сетки	13
1.4.2. Множества для классификации объектов и списки кнопок	14
1.4.3. Функция загрузки и поворота изображения.....	14
1.5. Шаг 2 — Класс Grid	15
1.5.1. Инициализация.....	15
1.5.2. Отображение координат — mouseMoveEvent.....	16
1.5.3. Метод paintEvent — механизм отрисовки.....	16
1.5.4. Обработка клика мыши — mousePressEvent	16
1.5.5. Сохранение и загрузка — save() и load().....	17
1.6. Шаг 3 — Класс Side	17
1.6.1. Инициализация и компоновка	17
1.6.2. Метод _sec() — создание секции с кнопками	18
1.6.3. Выбор объекта — метод _pick()	18
1.6.4. Переключение режимов — метод switch().....	18
1.6.5. Панель свойств — метод show_props().....	19
1.7. Шаг 4 — Класс App	20
1.7.1. Диалоги выбора файла	20
1.7.2. Точка входа в приложение.....	21
1.8. Типичные ошибки и их анализ.....	21
1.8.1. Программа сразу закрывается.....	21
1.8.2. Изображения не отображаются	21

1.8.3. Ошибка при загрузке — KeyError или ValueError	22
1.8.4. Объект не ставится на клетку	22
1.8.5. Поворот не применяется	22
1.9. Краткий справочник PyQt6.....	22
1.9.1. Основные виджеты.....	22
1.9.2. Компоновщики.....	22
1.9.3. Графические инструменты.....	23
1.9.4. Ключевые методы	23
ГЛАВА 2. РАСШИРЕНИЕ: БАЗА ДАННЫХ И ARDUINO.....	24
2.1. Введение к главе	24
2.1.1. Дополнительные импорты и зависимости.....	24
2.1.2. Расширенные константы.....	24
2.2. Журналирование событий — SQLite	25
2.2.1. Инициализация базы данных — функция init_db().....	25
2.2.2. Функция записи события — log().....	25
2.3. Взаимодействие с Arduino — класс Arduino.....	26
2.3.1. Инициализация соединения.....	26
2.3.2. Отправка команды — метод send_tl()	26
2.3.3. Глобальный объект и диалог выбора порта	26
2.4. Расширение класса Side — управление светофором	27
2.4.1. Расширенный __init__ класса Side.....	27
2.4.2. Ручной режим светофора.....	27
2.4.3. Автоматический режим светофора.....	28
2.4.4. Метод _auto_tick()	28
2.5. Скетч Arduino.....	29
2.5.1. Распиновка и функция управления.....	29
2.5.2. Функции setup() и loop()	29
2.5.3. Схема подключения	30
2.6. Типичные ошибки.....	30
2.6.1. Ошибки при работе с базой данных	30
2.6.2. Arduino не реагирует на команды.....	30
2.6.3. Таймер продолжает работать после удаления объекта.....	30
2.7. Краткий справочник	31
2.7.1. Модуль sqlite3	31
2.7.2. Библиотека pyserial	31
2.7.3. QTimer	32
2.7.4. Язык C++ для скетча Arduino	32

ГЛАВА 3. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYQT5.....	33
3.1. Инициализация последовательного подключения и работа с Arduino	33
3.2. Класс объектов сцены и система отображения.....	34
3.3. Привязка объектов к сетке.....	36
3.4. Обработка размещения объектов на сцене	36
3.5. Формирование и обновление пользовательского интерфейса	37
3.6. Создание панели инструментов.....	38
3.7. Панель свойств объектов	39
3.8. Панель управления режимами работы системы	40
3.9. Автоматическое управление светофорами	41
3.10. Формирование панели свойств объектов.....	42
3.11. Сохранение и загрузка сцены.....	44
3.12. Автоматический режим управления движением	46
3.13. Обработка аварийного режима.....	47
3.14. Ключевые функции в SQLite.....	47
3.15. Список использованных источников	48
ГЛАВА 4. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА pygame	49
4.1. Шаг 1 — Константы и вспомогательный функции	49
4.1.1. Константы и подключение к Arduino UNO.....	49
4.1.2. Загрузка изображений.....	50
4.2. Шаг 2 — Класс Obj	51
4.2.1. Инициализация и отрисовка объектов класса Obj	51
4.2.2 Метод update_car().....	52
4.3. Шаг 3 — класс Editor	54
4.3.1. Инициализация компонентов класса Editor().....	54
4.3.2. Метод add_db_record.....	55
4.3.3. Методы send и check_arduino.....	56
4.3.4. Методы update и set_light – переключение светофоров.....	57
4.3.5. Метод draw_db_ui.....	58
4.3.6. Метод draw_ui – отрисовка пользовательского интерфейса	59
4.3.7. Методы save, load — сохранение и загрузка сцены.....	61
4.3.8. Вспомогательные функции размещения объектов на холсте.....	62
4.4. Шаг 4 — Обработка событий и игровой цикл	63
4.4.1. Обработка событий	64
4.4.2. Событие MOUSEBUTTONDOWN	66
4.4.3. Завершающие события.....	70
4.5. Взаимодействие с Arduino UNO.....	70

4.5.1. Константы и setup().....	71
4.5.2. Основной цикл loop().....	72
4.5.3. Функция setLight().....	73
4.6. Типичные ошибки программы на pygame	74
4.7. Краткий справочник Pygame	75
4.8. Ключевые функции в SQLite.....	75
ГЛАВА 5. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYSIDE6	77
5.1. Введение в PySide6	77
4.2. Шаг 1 — Создание главного окна	77
4.2.1. Настройка окна.....	77
4.2.2. Компоновка интерфейса	78
4.2.3. Сохранение и загрузка	78
4.3. Шаг 2 — Константы и утилиты.....	79
4.3.1. Константы.....	79
4.3.2. Утилиты.....	80
5.4. Шаг 3 — Сцена симуляции — класс World	81
5.4.1. Поиск и навигация по сцене	81
5.4.2. Обработка нажатия по сцене	82
5.4.3. Отрисовка сетки	82
5.4.4. Обновление сущностей по таймеру.....	83
5.4.5. Генерация случайного маршрута	83
5.5. Шаг 4 — Базовый класс сущности — Baseentity	84
5.5.1. Инициализация.....	84
4.5.2. Поведение по таймеру.....	85
5.5.3. Установка текстуры с автоматическим масштабированием.....	85
5.5.4. Обработка перемещения	85
5.5.5. Установка клеточной позиции.....	86
5.5.6. Валидация размещения	86
5.6. Шаг 5 — Сериализация и загрузка сцены – модуль persistence	86
5.6.1. Сериализаторы сущностей.....	87
5.6.2. Сериализация сущности.....	87
5.6.3. Десериализация сущности	88
5.6.4. Сохранение сцены	88
5.6.5. Загрузка сцены	88
ЗАКЛЮЧЕНИЕ	90
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ	92

ВВЕДЕНИЕ

Настоящее методическое пособие разработано в соответствии с требованиями Федерального государственного образовательного стандарта среднего профессионального образования по специальности 09.02.08 «Интеллектуальные Интегрированные Системы» и рабочей программой дисциплины МДК.03 «Участие в разработке приложений взаимодействия с интеллектуальными интегрированными системами».

Актуальность темы. В условиях цифровой трансформации промышленности и транспортной инфраструктуры возрастает потребность в специалистах, способных проектировать и реализовывать программные комплексы, объединяющие графические интерфейсы, базы данных и аппаратные контроллеры. Интеллектуальные транспортные системы представляют собой один из наиболее востребованных классов таких решений, требующих от разработчика комплексного понимания архитектурных принципов, событийного программирования и взаимодействия с внешними устройствами.

Цель пособия — сформировать у обучающихся профессиональные компетенции в области проектирования архитектуры программных приложений через практическую реализацию проекта «Интеллектуальная Дорожная Система» в нескольких технологических стеках.

Задачи методического пособия:

- обеспечить понимание фундаментальных принципов построения графических приложений на языке Python;
- раскрыть архитектурные паттерны разделения данных и представления, обработки событий и управления состоянием;
- продемонстрировать практические аспекты интеграции с реляционными базами данных (SQLite) и микроконтроллерами (Arduino);
- сформировать навыки сравнительного анализа различных фреймворков (PyQt6, PyQt5, PySide6, Pygame) и выбора оптимального инструмента для решения конкретной задачи;
- развить умения самостоятельной диагностики и устранения типовых ошибок разработки.

Структура пособия построена по модульному принципу и включает пять глав:

Глава	Содержание	Технологический стек
-------	------------	----------------------

1	Базовая реализация визуального редактора карты: сетка, слои объектов, панель свойств, сохранение в JSON	Python + PyQt6
2	Расширение функционала: журналирование событий в SQLite, управление физическим светофором через Arduino, режимы работы	Python + PyQt6 + SQLite + pyserial
3	Альтернативная реализация на базе QGraphicsScene: работа с графической сценой, привязка к сетке, панели инструментов	Python + PyQt5
4	Реализация на игровом движке: активный игровой цикл, процедурная отрисовка интерфейса, обработка событий	Python + Pygame
5	Реализация на официальных биндингах Qt: объектно-ориентированная архитектура, сериализация, навигация по сцене	Python + PySide6

Методический подход. Пособие не является пошаговой инструкцией с готовыми решениями. Его концептуальная основа — объяснение *принципов* и *архитектурных решений*, лежащих в основе каждой реализации. Такой подход позволяет студенту:

- осознанно выбирать технические решения, а не механически копировать код;
- адаптировать полученные знания к новым требованиям и задачам;
- развивать навыки инженерного мышления и самостоятельного проектирования.

Ожидаемые результаты. После изучения материала и выполнения практического задания обучающийся должен:

Знать:

- принципы событийно-ориентированного и циклического программирования графических интерфейсов;
- архитектурные паттерны разделения ответственности между компонентами приложения;
- механизмы сериализации данных и взаимодействия с внешними устройствами;
- особенности лицензирования и выбора между PyQt6, PySide6 и альтернативными фреймворками.

Уметь:

- проектировать структуру приложения с учётом требований к масштабируемости и поддерживаемости;
- реализовывать пользовательские интерфейсы с использованием виджетов, компоновщиков и графических примитивов;

- организовывать журналирование событий и работу с реляционными базами данных;
- настраивать обмен данными с микроконтроллерами через последовательный интерфейс;
- диагностировать и устранять типовые ошибки разработки.

Владеть:

- навыками работы с документацией фреймворков и библиотеками Python;
- методами отладки графических приложений и анализа логов;
- практиками рефакторинга и адаптации кода под различные технологические платформы.

Рекомендации по использованию. Пособие предназначено для самостоятельной работы студентов под руководством преподавателя. Рекомендуется следующий порядок изучения:

1. Ознакомиться с функциональными требованиями и архитектурными принципами (Глава 1).
2. Реализовать базовую версию приложения на PyQt6.
3. Расширить функционал модулем журналирования и интеграцией с Arduino (Глава 2).
4. Для углубления понимания — сравнить реализацию на альтернативных фреймворках (Главы 3–5).
5. При возникновении трудностей обращаться к разделам «Типичные ошибки» и «Краткий справочник» в конце каждой главы.

Требования к программному обеспечению:

- Интерпретатор Python версии 3.10 или выше;
- Библиотеки: PyQt6 / PyQt5 / PySide6 / pygame (в зависимости от выбранной реализации);
- Дополнительные зависимости: pyserial, sqlite3 (входит в стандартную поставку Python);
- Среда разработки: Visual Studio Code, PyCharm или аналогичная;
- Для работы с Arduino: Arduino IDE, плата Arduino UNO или совместимая.

Материал пособия соответствует требованиям демонстрационного экзамена по специальности 09.02.08 и может использоваться как для аудиторных занятий, так и для самостоятельной подготовки обучающихся.

ГЛАВА 1. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYQT6

1.1. ВВЕДЕНИЕ К ГЛАВЕ

Перед тем как приступить к написанию кода, необходимо чётко понять, что именно предстоит создать и по каким принципам функционирует программа. Только понимание конечного образа позволяет принимать обоснованные технические решения на каждом этапе разработки.

Разрабатываемое приложение «Интеллектуальная Дорожная Система» представляет собой визуальный редактор карты. Пользователь работает с сеткой, на которую можно укладывать дорожное полотно и размещать поверх него объекты инфраструктуры: светофоры, знаки, пешеходные переходы, транспортные средства. Каждый элемент обладает набором свойств: он может быть повернут, заменён на другой вид или удалён. Готовая сцена сохраняется в файл формата JSON, откуда её можно загрузить в следующем сеансе работы.

С точки зрения архитектуры программирования подобные задачи относятся к классу редакторов с двумерным слоистым представлением данных. Это означает, что одни данные (дороги) лежат «ниже» других (объектов), и при отрисовке необходимо строго соблюдать порядок слоёв, чтобы элементы правильно перекрывали друг друга.

Функциональные требования

По условию задания программа должна реализовывать следующие возможности:

- главное окно с сеткой 21×21 клетку;
- боковая панель с переключением между режимами «Редактор дороги» и «Добавить объекты»;
- размещение изображений дорог на произвольные клетки сетки;
- размещение объектов инфраструктуры поверх дорог (или на свободные клетки для отдельных объектов);
- панель свойств выбранного объекта: поворот, смена вида, удаление;
- сохранение и загрузка текущей сцены в формате JSON.

Примечание. Все графические ресурсы (изображения .png и иконки .ico) находятся в папке *media/*, входящей в архив *Прил_ОЗ_КОД*. Создавать или изменять эти файлы самостоятельно не следует.

1.2. ПОДГОТОВКА РАБОЧЕЙ СРЕДЫ

1.2.1. Установка зависимостей

Прежде чем приступить к написанию кода, необходимо убедиться, что в системе установлен интерпретатор Python версии 3.10 и выше, а также сторонняя библиотека PyQt6. Понимание того, зачем нужна эта библиотека, принципиально важно: PyQt6 предоставляет Python-привязки к фреймворку Qt, что позволяет создавать полноценные графические интерфейсы с высокой производительностью отрисовки, обработкой событий мыши и клавиатуры, а также управлением компоновкой виджетов.

Стандартная библиотека Python не содержит инструментов для создания оконных GUI-приложений такого уровня: модуль tkinter слишком ограничен для задач с кастомной отрисовкой и слоями изображений. PyQt6 решает эту проблему, предоставляя полный цикл работы с графическими объектами — от рисования пикселей до обработки диалогов сохранения файлов.

Установка выполняется через менеджер пакетов `pip` одной командой:

```
pip install PyQt6
```

Определение библиотек для PyQt6:

```
import sys, json, os
from PyQt6.QtWidgets import *
from PyQt6.QtGui import QPainter, QColor, QPen, QIcon, QPixmap, QTransform
from PyQt6.QtCore import Qt
```

После установки следует убедиться, что структура проекта соответствует ожидаемой. Это важно, поскольку пути к ресурсам в коде указаны относительно главного файла:

```
module1/
|
|   main.py           # единственный файл с кодом приложения
|
|   media/            # папка с графическими ресурсами
|       Cbottom.png
|       TLred.png
|       ...           # остальные изображения из архива
|   run.bat           # скрипт запуска для Windows
```

1.2.2. Файл запуска run.bat и принцип рабочего каталога

Один из ключевых источников ошибок при работе с файловыми путями — несовпадение рабочего каталога (working directory) с директорией, в которой находится исполняемый файл. Когда Python-скрипт запускается двойным кликом, Windows по умолчанию устанавливает рабочим каталогом не папку со скриптом, а папку запуска интерпретатора. Это приводит к тому, что относительные пути вида `media/image.png` перестают работать.

Файл `run.bat` решает эту проблему: директива `cd /d "%~dp0"` принудительно переключает рабочий каталог на папку, где находится сам bat-файл. Символ `%~dp0` — это встроенная переменная Windows, раскрывающаяся в полный путь к директории bat-файла.

***Примечание.** Команда `pause` в конце bat-файла не даёт окну терминала закрыться сразу после запуска. Это позволяет увидеть сообщения об ошибках, если программа завершилась аварийно.*

1.3. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Прежде чем изучать принцип функционирования приложения, необходимо понять его общую архитектурную идею. Архитектура — это распределение ответственности между компонентами: какой класс что знает, чем управляет и с кем взаимодействует.

Приложение построено по принципу разделения данных и представления. Это один из фундаментальных принципов разработки графических приложений: модель (данные) существует отдельно от виджета (отображения), а виджет только читает данные и рисует их. Когда данные меняются, виджет перерисовывает себя заново.

1.3.1. Три основных класса

Всё приложение состоит из трёх классов, каждый из которых наследуется от одного из базовых виджетов PyQt6. `Grid(QWidget)` — сетка: хранит данные обоих слоёв, отрисовывает карту, обрабатывает клики мыши, это центральный компонент приложения. `Side(QWidget)` — нижняя панель: отображает кнопки выбора объектов,

переключает режимы работы, показывает панель свойств. `App(QMainWindow)` — главное окно: объединяет `Grid` и `Side` в единый интерфейс, добавляет панель управления.

1.3.2. Двухслойная модель данных

Ключевое архитектурное решение — хранение объектов в двух отдельных словарях Python. Это осознанное проектное решение, обусловленное требованиями задачи: словарь `roads` — нижний слой, дорожное полотно; словарь `objs` — верхний слой, объекты инфраструктуры, которые рисуются поверх дорог. Ключом каждого словаря служит кортеж (x, y) — координаты клетки.

1.3.3. Порядок инициализации и взаимные ссылки

Обратите внимание на порядок создания объектов в классе `App`. `Grid` при инициализации принимает ссылку на `Side`, поскольку при клике по объекту на сетке необходимо вызвать метод панели для отображения свойств. `Side`, в свою очередь, хранит ссылку на `Grid` для изменения его данных. Ситуация взаимной зависимости разрешается через двухэтапную инициализацию — паттерн «отложенная инициализация» (late binding).

```
self.side = Side()           # создаём Side первым
self.grid = Grid(self.side)  # Grid получает ссылку на Side
self.side.grid = self.grid   # Side получает обратную ссылку на Grid
```

1.4. ШАГ 1 — КОНСТАНТЫ И ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ

Профессиональное решение начинается с вынесения всех «чисел» и строковых литералов в именованные константы. Константа — это переменная, которая не меняется в процессе работы программы. Использование констант преследует несколько важных целей: устранение дублирования, повышение читаемости и упрощение изменений.

1.4.1. Импорт и параметры сетки

В начале файла подключаются все необходимые модули, после чего объявляются три константы, определяющие геометрию всей сетки. Изменение любой из них немедленно отражается на всех вычислениях, использующих эту константу.

```

import sys, json, os
from PyQt6.QtWidgets import *
from PyQt6.QtGui import QPainter, QColor, QPen, QIcon, QPixmap, QTransform
from PyQt6.QtCore import Qt, QSize

M = r"module1\media"
LOGO = f"{M}\\TLgreen.png"
IMG1, IMG2, IMG3, IMG4, IMG5 = f"{M}\\Cbottom.png",
f"{M}\\Pedestrian.png", f"{M}\\Block.png", f"{M}\\Zhorizontal.png", f"{M}\\TLyellow.png"
RD1, RD2 = f"{M}\\Rvertical.png", f"{M}\\Rcrossroads.png"

CELL, COLS, ROWS = 36, 21, 21

```

Значение $CELL = 36$ пикселей выбрано как компромисс: клетка достаточно большая, чтобы изображения были различимы, и достаточно маленькая, чтобы поле 21×21 вместились на экране без прокрутки ($21 \times 36 = 756$ пикселей).

1.4.2. Множества для классификации объектов и списки кнопок

Помимо числовых констант, объявляются списки для кнопок панели и множества (set) Python, которые определяют свойства объектов. Множество поддерживает проверку принадлежности за $O(1)$ — мгновенно, независимо от его размера.

```

PLACE_IMGS = [IMG1, IMG2, IMG3, IMG4, IMG5, RD1, RD2]
PLACE_NAMES = ["Машина", "Пешеход", "Знак", "Зебра", "Светофор", "Дорога", "Перекрёсток"]

FREE = {IMG2}
ROT = {IMG1, IMG2, IMG4, IMG5, RD1, RD2}
CYCLES = {
    IMG1: [IMG1, f"{M}\\BCvertical.png", f"{M}\\GCicon.ico"],
    IMG3: [IMG3, f"{M}\\Stop.png", f"{M}\\Start.png"],
    IMG5: [IMG5, f"{M}\\TLred.png", f"{M}\\TLgreen.png"],
}

```

Словарь CYCLES задаёт «карусельное» поведение: при каждом нажатии кнопки «Изменить тип» путь к изображению переключается на следующий элемент списка. Когда достигнут конец списка, следующим становится первый элемент. Реализуется это через операцию взятия остатка от деления (%).

1.4.3. Функция загрузки и поворота изображения

Функция `get_px()` инкапсулирует два связанных действия: загрузку изображения из файла и его поворот при необходимости. Вынесение этой логики в отдельную функцию устраняет дублирование, поскольку загрузка с поворотом происходит в нескольких местах программы.

```
def get_px(path, rot=0):  
    p = QPixmap(path)  
    return p.transformed(QTransform().rotate(rot)) if rot and not p.isNull() else p
```

`QPixmap` — класс `PyQt6` для хранения растровых изображений в памяти, оптимизированный для отрисовки на экране. `QTransform` — объект, описывающий геометрическое преобразование. Метод `transformed()` возвращает новый `QPixmap` с применённым преобразованием.

Примечание. Метод `isNull()` проверяет, был ли файл успешно загружен. Если файл не найден или повреждён, `QPixmap` возвращает «нулевой» объект. Попытка нарисовать нулевой `QPixmap` приведёт к пустому месту, но не к аварийному завершению. Всегда проверяйте этот флаг перед отрисовкой.

1.5. ШАГ 2 — КЛАСС GRID

Класс `Grid` является центральным компонентом приложения. Именно он хранит все данные карты и отвечает за их визуальное представление.

1.5.1. Инициализация

`Grid` наследуется от `QWidget` — базового строительного блока всех визуальных элементов в Qt. Наследование обеспечивает `Grid` всеми возможностями `QWidget`: отрисовку, обработку событий мыши, встраивание в компоновщики. Вызов `super().__init__()` в конструкторе инициализирует родительский класс и является обязательным шагом. `setFixedSize()` устанавливает точный размер виджета в пикселях и запрещает его изменение.

```
class Grid(QWidget):  
    def __init__(self, side):  
        super().__init__()  
        self.setFixedSize(COLS * CELL, ROWS * CELL)  
        self.side = side; self.roads, self.objs = {}, {}; self.mode = self.sel = None  
        self.setMouseTracking(True)
```

1.5.2. Отображение координат — mouseMoveEvent

Метод `mouseMoveEvent` вызывается Qt при каждом движении мыши над виджетом. Он переводит пиксельные координаты курсора в координаты клетки и отображает их в виде всплывающей подсказки. Это вспомогательный элемент интерфейса, помогающий ориентироваться на сетке.

```
def mouseMoveEvent(self, e):  
    x, y = int(e.position().x()//CELL), int(e.position().y()//CELL)  
    self.setToolTip(f"x:{x} y:{y}" if 0<=x<COLS and 0<=y<ROWS else "")
```

1.5.3. Метод paintEvent — механизм отрисовки

Метод `paintEvent` вызывается автоматически системой Qt каждый раз, когда виджет нужно перерисовать: при первом показе, при восстановлении из-под другого окна или при явном вызове `self.update()`. Вызывать `paintEvent` напрямую недопустимо. Внутри него создаётся объект `QPainter` — «чертёжный инструмент». Порядок отрисовки принципиально важен: сначала `roads`, затем `objs` поперх, и только потом линии сетки.

```
def paintEvent(self, _):  
    p = QPainter(self); p.fillRect(self.rect(), QColor(235, 235, 235))  
    for layer in (self.roads, self.objs):  
        for (x, y), o in layer.items():  
            img = get_px(o["path"], o["rot"])  
            if not img.isNull(): p.drawPixmap(x*CELL, y*CELL, CELL, CELL, img)  
    p.setPen(QPen(QColor(0, 0, 0, 50)))  
    for i in range(COLS+1): p.drawLine(i*CELL, 0, i*CELL, ROWS*CELL)  
    for i in range(ROWS+1): p.drawLine(0, i*CELL, COLS*CELL, i*CELL)
```

1.5.4. Обработка клика мыши — mousePressEvent

Метод `mousePressEvent` вызывается Qt при каждом нажатии кнопки мыши. Первая задача — преобразовать координаты клика в пикселях в координаты клетки через целочисленное деление. Ветвление `if/elif` описывает конечный автомат: в режиме 'place' с выбранным объектом из `ROADS` — укладываем дорогу в `roads`; с другим объектом — размещаем в `objs` при наличии дороги или принадлежности к `FREE`. Без выбранного объекта — открываем панель свойств.

```

def mousePressEvent(self, e):
    if e.button() != Qt.MouseButton.LeftButton: return
    x, y = int(e.position().x()//CELL), int(e.position().y()//CELL)
    if not (0 <= x < COLS and 0 <= y < ROWS): return
    c = (x, y)
    if self.mode == 'place' and self.sel:
        if self.sel in (RD1, RD2): self.roads[c] = {"path": self.sel, "base": self.sel, "rot": 0}
        elif c in self.roads or self.sel in FREE: self.objs[c] = {"path": self.sel, "base": self.sel,
"rot": 0, "speed": 0}
    else:
        o = self.objs.get(c) or self.roads.get(c)
        if o: self.side.show_props(c, o, self)
    self.update()

```

1.5.5. Сохранение и загрузка — save() и load()

Словари Python с кортежными ключами несовместимы с форматом JSON напрямую: стандарт JSON допускает только строковые ключи. При сохранении кортеж (x, y) конвертируется в строку вида 'x,y' через f-строку; при загрузке выполняется обратное преобразование через split(',') с лямбда-функцией.

```

def save(self, path):
    with open(path, "w", encoding="utf-8") as f:
        json.dump({"roads": {f"{k[0]},{k[1]}": v for k,v in self.roads.items()},
                  "objs": {f"{k[0]},{k[1]}": v for k,v in self.objs.items()}}, f, indent=2)

def load(self, path):
    try:
        with open(path, encoding="utf-8") as f: data = json.load(f)
        p = lambda d: {tuple(map(int, k.split(","))): v for k,v in d.items()}
        self.roads, self.objs = p(data.get("roads", {})), p(data.get("objs", {})); self.update()
    except Exception: pass

```

1.6. ШАГ 3 — КЛАСС SIDE

Класс Side управляет пользовательским взаимодействием: отображает доступные объекты в виде текстовых кнопок, переключает режимы работы и показывает панель свойств выбранного объекта. В отличие от традиционной «боковой панели», Side располагается снизу — над кнопками управления.

1.6.1. Инициализация и компоновка

Side наследуется от QWidget и использует горизонтальный компоновщик QHBoxLayout. Секция кнопок объектов и секция свойств по умолчанию скрыты методом hide() — они показываются только в нужный момент. Это паттерн

«переключаемых панелей». Атрибут `self.grid` изначально равен `None` и устанавливается позже из класса `App` (паттерн `late binding`).

```
class Side(QWidget):
    def __init__(self):
        super().__init__()
        self.grid = None; self.ibtns = []

        hb = QHBoxLayout(self); hb.setContentsMargins(6,2,6,2); hb.setSpacing(4)
        self.sp = self._sec(PLACE_IMGS, PLACE_NAMES)

        self.props = QWidget(); self.pvb = QHBoxLayout(self.props)
        self.pvb.setContentsMargins(0,0,0,0); self.pvb.setSpacing(4)
        for w in (self.sp, self.props): w.hide(); hb.addWidget(w)
        hb.addStretch()
```

1.6.2. Метод `_sec()` — создание секции с кнопками

Метод `_sec()` — пример паттерна «фабричный метод»: он принимает список путей к изображениям и список названий, возвращая готовый виджет с набором текстовых кнопок. Каждая кнопка создаётся с атрибутом `setCheckable(True)`, что переводит её в режим переключателя. Метод `setProperty()` хранит путь к изображению внутри кнопки.

```
def _sec(self, paths, names):
    w = QWidget(); hb = QHBoxLayout(w); hb.setContentsMargins(0,0,0,0); hb.setSpacing(4)
    for p, name in zip(paths, names):
        b = QPushButton(name); b.setFixedHeight(30); b.setCheckable(True); b.setProperty("path", p)
        b.clicked.connect(self._pick); hb.addWidget(b); self.ibtns.append(b)
    return w
```

1.6.3. Выбор объекта — метод `_pick()`

При клике на кнопку вызывается метод `_pick()`. Его задача — реализовать поведение «радио-кнопок»: одновременно может быть активна только одна. Ключевой инструмент — `self.sender()`: этот метод возвращает виджет, который отправил сигнал, позволяя одному обработчику обслуживать множество кнопок.

```
def _pick(self):
    s = self.sender()
    for b in self.ibtns:
        if b is not s: b.setChecked(False)
    if self.grid: self.grid.sel = s.property("path") if s.isChecked() else None
```

1.6.4. Переключение режимов — метод `switch()`

Метод `switch()` управляет видимостью секций и текущим режимом сетки. Именованный аргумент `place` позволяет вызывать метод интуитивно: `side.switch(place=True)` для включения режима размещения. При выключении — сбрасывает выбор кнопок, снимает выбранный объект и скрывает обе секции.

```
def switch(self, place=False):
    for b in self.ibtns: b.setChecked(False)
    if self.grid: self.grid.sel = None; self.grid.mode = 'place' if place else None
    self.props.hide(); self.sp.setVisible(place)
```

1.6.5. Панель свойств — метод `show_props()`

Метод `show_props()` реализует динамическое создание интерфейса: кнопки в панели свойств создаются заново каждый раз при выборе объекта. Их состав определяется типом объекта (поле `base`). Перед добавлением новых кнопок предыдущее содержимое удаляется через `takeAt(0)` и `deleteLater()`. Панель свойств расположена горизонтально (`QHBoxLayout`), что позволяет ей компактно отображаться в нижней полосе.

```
def show_props(self, cell, obj, grid):
    while self.pvb.count():
        w = self.pvb.takeAt(0).widget()
        if w: w.deleteLater()
    self.props.show(); base = obj["base"]
    self.pvb.addWidget(QLabel(f"<b>{os.path.basename(base)}</b>"))
    if base in ROT:
        b = QPushButton("Повернуть")
        b.clicked.connect(lambda: (obj.update(rot=(obj["rot"]+90)%360), grid.update()))
        self.pvb.addWidget(b)
    if base in CYCLES:
        b = QPushButton("Изменить тип"); c = CYCLES[base]
        def color(_, o=obj, cy=c):
            i = cy.index(o["path"]) if o["path"] in cy else 0
            o["path"] = cy[(i+1)%len(cy)]; grid.update()
        b.clicked.connect(color); self.pvb.addWidget(b)
    b = QPushButton("Удалить"); b.setStyleSheet("color:red;")
    def dele(_, c=cell, g=grid): g.objs.pop(c, None); g.roads.pop(c, None); g.update(); self.props.hide()
    b.clicked.connect(dele); self.pvb.addWidget(b)
```

Предупреждение. Обратите внимание на аргументы по умолчанию в определениях функций: `o=obj, cy=c, c=cell, g=grid`. Это фундаментальный приём Python при создании замыканий. Без него все функции захватили бы одну ссылку на переменную, значение которой к моменту вызова могло уже измениться.

1.7. ШАГ 4 — КЛАСС APP

Класс App является «оркестратором» приложения. Он наследуется от QMainWindow и объединяет Grid, Side и панель управления в единый интерфейс. Именно здесь задаётся компоновка: сетка располагается сверху, под ней — Side с кнопками объектов, под Side — панель с кнопками управления.

```
class App(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Интеллектуальная Дорожная Система")
        lp = QPixmap(LOGO)
        if not lp.isNull(): self.setWindowIcon(QIcon(lp))
        self.side = Side(); self.grid = Grid(self.side); self.side.grid = self.grid

        c = QWidget(); self.setCentralWidget(c)
        root = QVBoxLayout(c); root.setContentsMargins(0,0,0,0); root.setSpacing(0)
        root.addWidget(self.grid)

        bar = QHBoxLayout(); bar.setContentsMargins(6,6,6,6); bar.setSpacing(6)
        self.bp = QPushButton("Добавить объекты"); self.bp.setCheckable(True)
        bs = QPushButton("Сохранить"); bl = QPushButton("Загрузить")
        self.bp.toggled.connect(lambda v: self.side.switch(place=v))
        bs.clicked.connect(lambda: (fn:=QFileDialog.getSaveFileName(self, "", "", "JSON (*.json)")[0]) and self.grid.save(fn))
        bl.clicked.connect(lambda: (fn:=QFileDialog.getOpenFileName(self, "", "", "JSON (*.json)")[0]) and self.grid.load(fn))
        for b in (self.bp, bs, bl):
            b.setFixedHeight(32); b.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed); bar.addWidget(b)

        bottom = QVBoxLayout(); bottom.setContentsMargins(0,0,0,0); bottom.setSpacing(0)
        bottom.addWidget(self.side); bottom.addLayout(bar)
        root.addLayout(bottom); self.adjustSize()
```

Компоновщики QVBoxLayout и QHBoxLayout управляют расположением виджетов. Вертикальный корневой компоновщик root содержит сетку grid и вложенный bottom-компоновщик. Метод addLayout() позволяет вкладывать компоновщики друг в друга.

1.7.1. Диалоги выбора файла

QFileDialog предоставляет стандартный системный диалог выбора файла. Статические методы getSaveFileName() и getOpenFileName() возвращают кортеж (путь, фильтр). Если пользователь закрыл диалог без выбора, путь будет пустой строкой — отсюда необходимость проверки перед вызовом save()/load().

Сигнал `toggled` у кнопок-переключателей отличается от `clicked`: `toggled` посылается при изменении состояния и передаёт новое состояние (`True/False`) в подключённый слот. Использование `toggled` позволяет корректно обрабатывать программное изменение через `setChecked()`.

1.7.2. Точка входа в приложение

Конструкция `if __name__ == '__main__':` обозначает точку входа. Код внутри неё выполняется только при прямом запуске файла, но не при его импорте как модуля. `QApplication` — синглтон, управляющий приложением Qt; он должен быть создан до любого виджета. Метод `exec()` запускает цикл событий.

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = App()
    window.show()
    sys.exit(app.exec())
```

1.8. ТИПИЧНЫЕ ОШИБКИ И ИХ АНАЛИЗ

Умение диагностировать и устранять ошибки — профессиональный навык, не менее важный, чем написание кода. В данном разделе описаны наиболее распространённые проблемы и объясняется, как понять их причину самостоятельно.

1.8.1. Программа сразу закрывается

Если программа закрывается мгновенно, это означает, что Python завершил работу из-за необработанного исключения. Чтобы увидеть текст ошибки, добавьте в `run.bat` перенаправление вывода в файл:

```
@echo off
cd "ПОЛНЫЙ ПУТЬ К ФАЙЛУ ПРОГРАММЫ"
python main.py
pause
```

1.8.2. Изображения не отображаются

Наиболее частая причина — неверный путь к файлу. Следует использовать относительные пути от папки запуска, а не абсолютные пути конкретного компьютера. Для диагностики добавьте временную проверку: `print(QPixmap(IMG1).isNull())`. Если выводится `True` — файл не найден.

1.8.3. Ошибка при загрузке — KeyError или ValueError

Эта ошибка возникает, если формат ключей в JSON-файле не соответствует ожидаемому. Откройте JSON-файл в текстовом редакторе и убедитесь, что ключи словарей roads и objs имеют формат '0,5', '12,3' — два числа, разделённых запятой без пробелов и без скобок.

1.8.4. Объект не ставится на клетку

Необходимо одновременно выполнение двух условий: в переменной self.sel должен быть путь к выбранному файлу, и либо клетка присутствует в self.roads, либо объект входит в множество FREE, либо объект является дорогой (RD1, RD2). Для диагностики добавьте временный вывод в mousePressEvent:

```
print(f'mode={self.mode}, sel={self.sel}, in_roads={c in self.roads}')
```

1.8.5. Поворот не применяется

Убедитесь, что поле base объекта входит в множество ROT, а после изменения поля rot вызывается grid.update(). Без вызова update() Qt не знает об изменении данных и не запрашивает перерисовку.

1.9. КРАТКИЙ СПРАВОЧНИК PYQT6

В данном разделе перечислены наиболее употребимые классы и методы PyQt6, используемые в проекте. При углублённом изучении обращайтесь к официальной документации Qt (doc.qt.io).

1.9.1. Основные виджеты

QMainWindow — главное окно приложения, поддерживает меню и центральный виджет. QWidget — базовый строительный блок, любой визуальный элемент Qt является его наследником. QPushButton — кнопка с поддержкой режима переключателя и хранения данных через setProperty. QLabel — текстовая метка или контейнер для изображения. QFileDialog — системный диалог выбора файла для сохранения или открытия.

1.9.2. Компоновщики

QVBoxLayout — вертикальный компоновщик, виджеты располагаются один под другим. QHBoxLayout — горизонтальный компоновщик, виджеты располагаются слева

направо. Метод `addWidget(w)` добавляет виджет в компоновщик, `addLayout(l)` вкладывает один компоновщик в другой, `addStretch()` добавляет эластичный разделитель, заполняющий свободное пространство.

1.9.3. Графические инструменты

`QPainter` — инструмент рисования, используется только внутри `paintEvent()`. `QPixmap` — растровое изображение, оптимизированное для отрисовки. `QIcon` — иконка виджета, создаётся из `QPixmap`. `QTransform` — геометрическое преобразование: поворот, масштабирование, смещение. `QColor` — цвет в формате RGB или RGBA (с прозрачностью). `QPen` — перо для линий: цвет, толщина, стиль.

1.9.4. Ключевые методы

# Перерисовка	
<code>widget.update()</code>	# запросить перерисовку
# Размер и видимость	
<code>widget.setFixedSize(w, h)</code>	# фиксированный размер
<code>widget.setVisible(bool)</code>	# показать / скрыть
<code>widget.adjustSize()</code>	# подогнать под содержимое
# Кнопки	
<code>btn.setCheckable(True)</code>	# режим переключателя
<code>btn.setChecked(True/False)</code>	# состояние программно
<code>btn.setProperty('key', value)</code>	# сохранить данные в виджете
<code>btn.property('key')</code>	# прочитать данные
# Сигналы	
<code>btn.clicked.connect(func)</code>	# нажатие
<code>btn.toggled.connect(func)</code>	# изменение состояния
<code>self.sender()</code>	# виджет — источник сигнала
# Компоновщик	
<code>layout.count()</code>	# количество элементов
<code>layout.takeAt(0)</code>	# извлечь элемент по индексу
<code>item.widget()</code>	# получить виджет из элемента
<code>widget.deleteLater()</code>	# запланировать удаление из памяти

В первой главе методического пособия были рассмотрены фундаментальные принципы и практические шаги по созданию графического приложения «Интеллектуальная Дорожная Система» на базе фреймворка PyQt6. Ниже представлены ключевые итоги изучения материала:

1. Понимание предметной области и требований.
2. Подготовка рабочей среды.
3. Архитектурные принципы.

ГЛАВА 2. РАСШИРЕНИЕ: БАЗА ДАННЫХ И ARDUINO

Журналирование событий, управление физическим светофором.

2.1. ВВЕДЕНИЕ К ГЛАВЕ

Второй модуль является логическим продолжением первого: базовый визуальный редактор карты расширяется тремя новыми возможностями. Во-первых, все значимые события в программе начинают записываться в реляционную базу данных SQLite. Во-вторых, приложение приобретает способность управлять физическим светофором через Arduino. В-третьих, объект «Светофор» получает два режима управления: ручной и автоматический.

Структура кода модуля 2 полностью совпадает со структурой модуля 1. Классы называются так же: Grid, Side, App. Ниже описывается только новый функционал и изменения, которые нужно внести в уже существующий код.

2.1.1. Дополнительные импорты и зависимости

К строке импорта добавляются новые модули. Конструкция try/except при импорте pyserial — стандартный способ реализации необязательных зависимостей. Флаг HAS_SERIAL проверяется во всех местах, где используется библиотека serial.

```
import sys, json, os, sqlite3, time
from PyQt6.QtWidgets import *
from PyQt6.QtGui import QPainter, QColor, QPen, QIcon, QPixmap, QTransform
from PyQt6.QtCore import Qt, QTimer
try: import serial; HAS_SERIAL = True
except: HAS_SERIAL = False
```

Для установки pyserial выполните в терминале: `pip install pyserial`. Библиотеки `sqlite3`, `time` и `os` входят в стандартную поставку Python и не требуют установки.

2.1.2. Расширенные константы

К константам первого модуля добавляются пути к файлам базы данных и карты, а также словари для светофорной логики. Словарь `TL_CMD` связывает путь к изображению текущего состояния светофора с однобайтовой командой для Arduino. Словарь `OBJECTS` задаёт человекочитаемые названия всех объектов по их пути — используется в панели свойств.

```
MAP_FILE = r"module1\map.json"
DB_FILE = r"events.db"

TL_RED, TL_YELLOW, TL_GREEN = f"{M}\\TLred.png", f"{M}\\TLyellow.png", f"{M}\\TLgreen.png"
TL_CYCLE = [TL_RED, TL_YELLOW, TL_GREEN]
TL_CMD = {TL_RED: b"R", TL_YELLOW: b"Y", TL_GREEN: b"G"}
ROADS = {RD1, RD2}
OBJECTS = {IMG1:"Машина", IMG2:"Пешеход", IMG3:"Знак", IMG4:"Зебра",
           IMG5:"Светофор", RD1:"Дорога", RD2:"Перекрёсток"}
```

2.2. ЖУРНАЛИРОВАНИЕ СОБЫТИЙ — SQLITE

Журналирование — запись значимых событий в постоянное хранилище — является стандартной практикой в профессиональной разработке. В данном проекте роль хранилища выполняет SQLite — встроенная в Python реляционная СУБД. Вся база данных хранится в одном файле events.db рядом с программой.

2.2.1. Инициализация базы данных — функция `init_db()`

Функция `init_db()` вызывается один раз при запуске программы. Она создаёт файл базы данных (если он не существует) и гарантирует наличие таблицы events. Ключевое слово IF NOT EXISTS предотвращает ошибку при повторном запуске. Эту функцию добавляем в начало файла, перед определением классов.

```
def init_db():
    con = sqlite3.connect(DB_FILE)
    con.execute("CREATE TABLE IF NOT EXISTS events (id INTEGER PRIMARY KEY AUTOINCREMENT, time TEXT,
    event TEXT)")
    con.commit(); con.close()
```

2.2.2. Функция записи события — `log()`

Функция `log()` открывает соединение с базой, записывает одну строку и немедленно закрывает соединение. Параметрический запрос с символами ? вместо прямой подстановки строки является обязательной практикой: он защищает от SQL-инъекций. Функцию добавляем сразу после `init_db()`.

```
def log(event):
    con = sqlite3.connect(DB_FILE)
    con.execute("INSERT INTO events(time,event) VALUES(?,?)", (time.strftime("%Y-%m-%d %H:%M:%S"), event))
    con.commit(); con.close()
```

2.3. ВЗАИМОДЕЙСТВИЕ С ARDUINO — КЛАСС ARDUINO

Класс Arduino инкапсулирует всю логику работы с физическим устройством через последовательный порт. Его конструктор принимает строку с именем порта (например, 'COM3') и пытается установить соединение. Класс добавляем в файл после функции log(), до определения Grid.

2.3.1. Инициализация соединения

Попытка подключения обернута в try/except, что делает класс устойчивым к любым проблемам с портом. При неудаче объект serial остаётся None, и последующие вызовы send_tl() просто ничего не делают. Вызов time.sleep(2) обязателен: большинство плат Arduino при подключении по USB автоматически перезагружаются.

```
class Arduino:
    def __init__(self, port):
        self._s = None
        if HAS_SERIAL and port:
            try: self._s = serial.Serial(port, 9600, timeout=1); time.sleep(2)
            except Exception as e: print(f"Arduino: {e}")
```

2.3.2. Отправка команды — метод send_tl()

Метод send_tl() принимает путь к текущему изображению светофора, находит соответствующую однобайтовую команду в словаре TL_CMD и отправляет её в последовательный порт. Операция flush() гарантирует, что данные немедленно покидают буфер операционной системы. Конструкция if cmd := использует оператор «морж» (:=), введённый в Python 3.8.

```
def send_tl(self, path):
    if cmd := TL_CMD.get(path):
        try: self._s and self._s.write(cmd) and self._s.flush()
        except Exception as e: print(f"Serial: {e}")

ARD = None
```

2.3.3. Глобальный объект и диалог выбора порта

Объект класса Arduino создаётся в точке входа как глобальная переменная ARD. Перед запуском главного окна пользователю предлагается ввести имя COM-порта через стандартный диалог QInputDialog. Если пользователь закрыл диалог или оставил поле

пустым, Arduino создаётся с `port=None` и программа работает без физического устройства.

Кроме того, в точку входа добавляется автоматическая загрузка карты из `MAP_FILE` при запуске через `QTimer.singleShot`, а также вызов `init_db()` для инициализации базы данных.

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    init_db()
    port, ok = QInputDialog.getText(None, "Arduino", "COM-порт\nПример: COM3")
    ARD = Arduino(port.strip() if ok and port.strip() else None)
    w = App(); w.show()
    QTimer.singleShot(0, lambda: (w.grid.load(MAP_FILE), w.grid.update()))
    sys.exit(app.exec())
```

2.4. РАСШИРЕНИЕ КЛАССА SIDE — УПРАВЛЕНИЕ СВЕТОФОРОМ

Второй модуль существенно расширяет метод `show_props()` класса `Side`: панель свойств теперь умеет работать с таймером для автоматического режима, отправлять команды Arduino при изменении состояния светофора и записывать каждое переключение в журнал базы данных.

В класс `Side` добавляются два атрибута в `__init__`: `self._timer` и `self._auto_obj`, `self._auto_grid` — для хранения состояния авторежима.

2.4.1. Расширенный `init` класса `Side`

В конструктор `Side` добавляются поля для хранения объекта и сетки, к которым относится текущий авторежим, а также объект `QTimer`. `QTimer` — объект Qt, генерирующий сигнал `timeout` с заданным интервалом без блокировки интерфейса.

```
# добавить в __init__ после self.ibtns = []:
self._auto_obj = None
self._auto_grid = None
self._timer = QTimer(interval=3000)
self._timer.timeout.connect(self._auto_tick)
```

2.4.2. Ручной режим светофора

В методе `show_props()` добавляем блок для светофора (`base == IMG5`). При каждом нажатии кнопки «Ручной режим» состояние светофора переключается на

следующий элемент цикла TL_CYCLE. Одновременно команда отправляется в Arduino и событие записывается в базу данных.

```

    if base == IMG5:
        b = QPushButton("Ручной режим")
        def on_manual(_, o=obj, g=grid):
            idx = TL_CYCLE.index(o["path"]) if o["path"] in TL_CYCLE else 0
            o["path"] = TL_CYCLE[(idx+1)%3]
            if ARD: ARD.send_tl(o["path"])
            log(f'Ручной: {os.path.basename(o["path"])}'); g.update()
        b.clicked.connect(on_manual); self.pvb.addWidget(b)
k)

```

2.4.3. Автоматический режим светофора

Кнопка «Авторежим» является переключателем (checkable). При активации она запускает таймер и сохраняет ссылки на текущий объект и сетку. При деактивации — останавливает таймер. Оба действия записываются в журнал. Этот блок добавляется следом за кнопкой ручного режима.

```

ba = QPushButton("Авторежим"); ba.setCheckable(True)
def on_auto(checkered, o=obj, g=grid):
    if checked: self._auto_obj=o; self._auto_grid=g; self._timer.start(); log("Авторежим вкл")
    else: self._timer.stop(); log("Авторежим выкл")
ba.clicked.connect(on_auto); self.pvb.addWidget(ba)

```

2.4.4. Метод auto_tick()

Метод _auto_tick() вызывается таймером каждые 3 секунды. Он переключает состояние светофора по кольцевому циклу, запрашивает перерисовку сетки, отправляет команду Arduino и записывает событие в журнал. Ветка else является защитной: если path оказался вне TL_CYCLE, цикл корректно возвращается в начало. Метод добавляется в класс Side после show_props().

```

def _auto_tick(self):
    if not self._auto_obj: return
    o = self._auto_obj
    o["path"] = TL_CYCLE[(TL_CYCLE.index(o["path"])+1)%3] if o["path"] in TL_CYCLE else TL_CYCLE[0]
    if self._auto_grid: self._auto_grid.update()
    if ARD: ARD.send_tl(o["path"])
    log(f'Авто: {os.path.basename(o["path"])}')

```

2.4.5. Диагностика и Восстановление

Найдя блок `if base == IMG5`: внутри метода `show_props`. В самом конце этого блока, перед закрывающей скобкой, реализовать алгоритм диагностики и восстановления, он добавляет две кнопки которые появляются при клике на светофор. Диагностика выполняет проверку светодиодов, передает сигналы на аппаратный модуль. Восстановление выполняет перевод светодиода в желтый исходный цвет, а также передает сигналы на аппаратный модуль.

```
bd = QPushButton("Диагностика")
bd.clicked.connect(lambda _, o=obj, g=grid: on_manual(None, o, g))
self.pvb.addWidget(bd)

br = QPushButton("Восстановление")
br.clicked.connect(lambda _, o=obj, g=grid: (o.update(path=TL_YELLOW), ARD and ARD.send_t1(TL_YELLOW), g.update()))
self.pvb.addWidget(br)
```

2.4.6. Кнопка "Начать обучение"

В классе `App.__init__` строка где создаются кнопки `bs = QPushButton("Сохранить"); bl = QPushButton("Загрузить")`, после неё реализовать параметры создания кнопки. В дальнейшем здесь реализуется алгоритм машинного обучения .

```
bml = QPushButton("Начать обучение")
bml.clicked.connect(lambda: QMessageBox.information(self, "Обучение", "Здесь можно добавить алгоритм машинного обучения"))
```

2.4.7. Кнопка объявление кнопки "Начать обучение"

В существующем `for b in (self.bp, bs, bl)`: добавить `bml` в конец кортежа.

```
for b in (self.bp, bs, bl, bml):
```

Примечание. Таймер необходимо останавливать при удалении объекта светофора с карты. В методе `show_props()` убедитесь, что функция `dele` содержит `self.timer.stop()`, иначе `_auto_tick()` продолжит обращаться к уже удалённому объекту.

2.5. СКЕТЧ ARDUINO

Скетч (программа) для микроконтроллера Arduino пишется на языке C++ с использованием стандартной библиотеки Arduino. Задача скетча предельно проста: ожидать байт в последовательном порту и в зависимости от принятого символа включать один из трёх светодиодов.

2.5.1. Распиновка и функция управления

Три константы PIN_R, PIN_Y, PIN_G определяют номера цифровых выводов Arduino, к которым подключены светодиоды красного, жёлтого и зелёного цветов. Функция setTL() принимает символ и устанавливает соответствующий вывод в HIGH, одновременно сбрасывая остальные в LOW. Выражение c == 'R' в C++ возвращает 1 (true) или 0 (false), что соответствует HIGH/LOW. После переключения светодиода функция отправляет подтверждение обратно в Python через Serial.println(), что позволяет программе убедиться что команда была получена и исполнена.

```
const int PIN_R = 8, PIN_Y = 9, PIN_G = 10;
const int BTN1 = 1, BTN2 = 2;

void setTL(char c) {
    digitalWrite(PIN_R, c == 'R');
    digitalWrite(PIN_Y, c == 'Y');
    digitalWrite(PIN_G, c == 'G');
    Serial.println(c);
}
```

2.5.2. Функции setup()

Функция setup() вызывается один раз при включении питания. Она настраивает пины светодиодов на вывод (OUTPUT), пины кнопок на вход с подтяжкой (INPUT_PULLUP) и инициализирует последовательный порт со скоростью 9600 бод — она должна совпадать со значением в Python-коде. После инициализации сразу вызывается setTL('Y'), чтобы светофор при включении всегда начинал с жёлтого цвета — это соответствует поведению кнопки "Восстановление" в программе.

```
void setup() {
    Serial.begin(9600);
    pinMode(PIN_R, OUTPUT); pinMode(PIN_Y, OUTPUT); pinMode(PIN_G, OUTPUT);
    pinMode(BTN1, INPUT_PULLUP); pinMode(BTN2, INPUT_PULLUP);
    setTL('Y');
}
```

2.5.3. Функция loop() — приём команд из Python

Функция loop() вызывается непрерывно в бесконечном цикле пока устройство включено. Первый блок внутри loop() проверяет наличие данных в буфере последовательного порта. Если Python отправил символ R, Y или G — вызывается setTL() и светофор переключается в соответствующий цвет. Это обеспечивает управление светофором из программы через кнопки "Ручной режим", "Диагностика" и "Восстановление".

```
void loop() {  
    if (Serial.available() > 0) {  
        char c = Serial.read();  
        if (c == 'R' || c == 'Y' || c == 'G') setTL(c);  
    }  
}
```

2.5.4. Функция loop() — обработка физических кнопок

Второй и третий блоки loop() обрабатывают нажатия физических кнопок. При срабатывании кнопки сначала выполняется задержка 50 мс, после чего состояние кнопки проверяется повторно — это программный антидребезг, исключающий ложные срабатывания от механических колебаний контактов. Если кнопка всё ещё нажата, Arduino отправляет строку "B1" или "B2" в Python, после чего выдерживается пауза 500 мс для защиты от повторного срабатывания. Python принимает эти строки и выполняет соответствующее действие: B1 — следующий цвет (Диагностика), B2 — жёлтый (Восстановление).

```
if (digitalRead(BTN1) == LOW) {  
    delay(50);  
    if (digitalRead(BTN1) == LOW) { Serial.println("B1"); delay(500); }  
}  
if (digitalRead(BTN2) == LOW) {  
    delay(50);  
    if (digitalRead(BTN2) == LOW) { Serial.println("B2"); delay(500); }  
}  
}
```

2.6. ТИПИЧНЫЕ ОШИБКИ

2.6.1. Ошибки при работе с базой данных

Если программа падает с ошибкой `OperationalError: unable to open database file` — убедитесь, что рабочий каталог установлен корректно (`run.bat` выполнен с `cd /d`). База данных `events.db` создаётся в текущем рабочем каталоге. Если таблица не найдена — убедитесь, что `init_db()` вызывается до создания `App()`.

2.6.2. Arduino не реагирует на команды

Проверьте следующее: скорость в `Serial.begin()` скетча совпадает с 9600 в Python; имя COM-порта введено корректно (COM3, COM4 и т. д.); скетч успешно загружен в Arduino через Arduino IDE; между открытием порта и отправкой данных есть пауза 2 секунды (`time.sleep(2)` в конструкторе класса `Arduino`).

2.6.3. Таймер продолжает работать после удаления объекта

Это происходит, если в функции `delete` (удаление объекта) не вызывается `self._timer.stop()`. Исправление: в методе `show_props()` в определении `on_del` добавьте

вызов `self._timer.stop()` после `g.objs.pop(c, None)`. Без этого `_auto_tick()` продолжит менять состояние несуществующего объекта.

```
def dele(_, c=cell, g=grid):
    self._timer.stop()
    g.objs.pop(c, None); g.update(); self.props.hide()
```

2.7. КРАТКИЙ СПРАВОЧНИК

В данном разделе представлены классы и функции, добавленные во втором модуле. Справочник по базовым компонентам PyQt6 приведён в разделе 10 данного пособия.

2.7.1. Модуль `sqlite3`

Элемент	Описание
<code>sqlite3.connect(path)</code>	Открыть соединение с файлом базы данных. Если файл не существует — создаёт его.
<code>con.execute(sql, params)</code>	Выполнить SQL-запрос с параметрами (кортеж или список).
<code>con.commit()</code>	Сохранить изменения в файл.
<code>con.close()</code>	Закрыть соединение и освободить ресурсы.
CREATE TABLE IF NOT EXISTS	Создать таблицу только если она ещё не существует.
AUTOINCREMENT	Автоматически присваивать уникальный порядковый номер.

2.7.2. Библиотека `pyserial`

Метод	Описание
<code>serial.Serial(port, baud, timeout)</code>	Открыть соединение с последовательным портом.
<code>ser.write(bytes)</code>	Отправить байты в порт.
<code>ser.flush()</code>	Принудительно отправить содержимое буфера.
<code>Serial.available()</code>	Arduino: число байт в буфере приёма.
<code>Serial.read()</code>	Arduino: прочитать один байт из буфера.

Метод	Описание
Serial.begin(baud)	Arduino: инициализировать UART с заданной скоростью.

2.7.3. QTimer

Метод	Описание
QTimer(interval=N, timeout=slot)	Создать таймер с интервалом N мс и подключить слот.
timer.start()	Запустить таймер. Сигнал timeout посылается каждые N мс.
timer.stop()	Остановить таймер.
timer.isActive()	Возвращает True, если таймер работает.

2.7.4. Язык C++ для скетча Arduino

Конструкция	Описание
pinMode(pin, OUTPUT)	Объявить вывод как выход.
digitalWrite(pin, value)	Установить вывод в HIGH (1) или LOW (0).
Serial.begin(baud)	Инициализировать последовательный порт.
Serial.available()	Вернуть число байт в буфере.
Serial.read()	Прочитать один байт.
c == 'R'	Сравнение символов в C++; возвращает 1 или 0.

ГЛАВА 2. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYQT5

2.1. Инициализация последовательного подключения и работа с Arduino

Для взаимодействия программы с микроконтроллером Arduino реализован класс `SerialManager`. Данный класс отвечает за подключение к COM-порту, отправку команд и получение данных от микроконтроллера через последовательный интерфейс `Serial`.

При запуске программы пользователь вручную вводит номер COM-порта. Если введено только число, программа автоматически добавляет префикс COM, что упрощает запуск приложения.

Подключение выполняется с помощью библиотеки `pyserial`. Скорость передачи данных установлена на 9600 бод, что является стандартным значением для Arduino UNO и большинства совместимых плат.

В случае ошибки подключения выводится сообщение в консоль. Это позволяет избежать аварийного завершения программы при отсутствии подключения к микроконтроллеру.

```
ARDUINO_PORT = input("COM порт: ")
digitalWrite(greenPin, 1);
ARDUINO_PORT = ARDUINO_PORT if len(ARDUINO_PORT) > 2 else "COM"+ARDUINO_PORT
```

Конструктор класса `SerialManager` выполняет:

- создание объекта последовательного соединения;
- подключение к COM-порту;
- ожидание инициализации Arduino;
- обработку ошибок подключения.

```
class SerialManager:
    def __init__(self):
        self.ser = None
        try:
            self.ser = serial.Serial(ARDUINO_PORT, 9600, timeout=0.1)
            print(f"Подключено к {ARDUINO_PORT}")
            sleep(2)
        except Exception as e:
            print(f"Ошибка подключения к Arduino: {e}")
```

Функция `send_command` отвечает за отправку команд микроконтроллеру. Перед отправкой выполняется проверка, открыт ли COM-порт. Команда передаётся в виде строки, преобразованной в байтовый формат методом `encode()`.

```
def send_command(self, cmd):
    if self.ser and self.ser.is_open:
        try:
            self.ser.write(cmd.encode())
        except Exception as e:
            print(f"Ошибка отправки: {e}")
```

Функция `read_line` выполняет чтение данных из последовательного порта. Перед чтением проверяется наличие входящих данных через свойство `in_waiting`. Полученная строка декодируется из UTF-8.

```
def read_line(self):
    try:
        if self.ser.in_waiting > 0:
            return self.ser.readline().decode('utf-8', errors='ignore')
    except Exception as e:
        print(f"Ошибка чтения: {e}")
    return None
```

2.2. Класс объектов сцены и система отображения

Для хранения объектов дорожной сцены реализован класс `SceneObject`, наследуемый от `QGraphicsPixmapItem`. Каждый объект сцены представляет отдельный графический элемент, размещаемый на рабочем поле редактора.

Класс хранит:

- тип объекта;
- координаты;
- набор дополнительных свойств;
- текущее графическое состояние.

```
class SceneObject(QGraphicsPixmapItem):
    def __init__(self, x, y, obj_type, props=None):
        super().__init__()
        self.obj_type = obj_type
        self.props = props if props else {}
        self.setPos(x, y)
        self.setFlags(QGraphicsItem.ItemIsSelectable | QGraphicsItem.ItemIsMovable)
        self.update_visuals()
```

Метод `update_visuals` отвечает за выбор изображения объекта и обновление его визуального состояния. В зависимости от типа объекта загружается соответствующий файл изображения.

Для светофора дополнительно реализована отправка команды на Arduino. Каждому состоянию соответствует собственный код:

- красный – 3;
- жёлтый – 2;
- зелёный – 1.

```
state_map = {"red": '3', "yellow": '2', "green": '1'}
```

При изменении состояния светофора программа:

- обновляет изображение;
- отправляет сигнал на микроконтроллер;
- синхронизирует физическое и программное состояние.

```
if self.obj_type == "светофор":
    self.setZValue(-18)
    state = self.props.get("State", "red")
    if state in state_map:
        serial_manager.send_command(state_map[state])
    pix.load(media_dir + f"TL{state}")
```

Для остальных объектов сцены реализована аналогичная система выбора изображений:

- дорожные знаки;
- пешеход;
- пешеходный переход;
- дорога;
- перекрёсток.

Объекты размещаются на разных слоях сцены через `setZValue()`. Это позволяет корректно отображать пересечение элементов и управлять их визуальным приоритетом.

2.3. Привязка объектов к сетке

Для упрощения построения дорожной схемы реализована система автоматической привязки объектов к сетке.

Метод `itemChange` выполняет округление координат объекта до ближайшей клетки размером 35×35 пикселей. Благодаря этому все элементы сцены располагаются строго по сетке и не выходят за её границы.

```
def itemChange(self, change, value):
    if change == QGraphicsItem.ItemPositionChange:
        grid_size = 35
        value.setX(int(value.x() / grid_size) * grid_size)
        value.setY(int(value.y() / grid_size) * grid_size)
        return value
    return super().itemChange(change, value)
```

Такой подход:

- упрощает построение схемы;
- обеспечивает единый размер объектов;
- предотвращает наложение элементов;
- облегчает проверку допустимости размещения.

2.4. Обработка размещения объектов на сцене

Для отображения сцены используется класс `GridView`, наследуемый от `QGraphicsView`.

Данный класс отвечает за:

- обработку нажатий мыши;
- размещение объектов;
- проверку допустимости установки;
- взаимодействие с редактором сцены.

При нажатии мыши координаты курсора преобразуются в координаты сетки.

```
snapx = int(pos.x() // self.grid_size) * self.grid_size
snapy = int(pos.y() // self.grid_size) * self.grid_size
```

После вычисления координат выполняется проверка возможности размещения объекта.

```
if self.is_valid_placement(snapx, snapy, self.editor.selected_obj_type):
    self.editor.add_object(snapx, snapy, self.editor.selected_obj_type)
```

Функция `is_valid_placement` реализует правила размещения объектов на сцене.

В системе реализованы следующие ограничения:

- светофор можно установить только возле перекрёстка;
- дорожные знаки размещаются возле перекрёстков;
- пешеход размещается возле перехода;
- пешеходный переход может находиться только на дороге.

Для проверки используются функции:

- `is_on_road`;
- `is_near_crossroad`.

Все проверки выполняются путём обхода объектов сцены и сравнения их координат.

Для привязки объектов к сетке используется функция `mousePressEvent`

```
def mousePressEvent(self, event):
    super().mousePressEvent(event)
    for item in self.scene.selectedItems():
        pos = item.pos()
        item.setPos(int((pos.x()+5) // self.size)*self.size,
                    int((pos.y()+10) // self.size)*self.size)
```

2.5. Формирование и обновление пользовательского интерфейса

Главное окно приложения реализовано на основе класса RoadEditor, наследуемого от QMainWindow. Данный класс объединяет графическую сцену, панель инструментов, панель свойств объектов и элементы управления режимами работы системы.

В конструкторе класса создаётся объект сцены QGraphicsScene, который используется для хранения всех графических элементов дорожной схемы.

```
self.scene = QGraphicsScene()
self.scene.setSceneRect(0, 0, 1000, 800)
```

Для отображения сцены используется пользовательский класс GridView, отвечающий за работу с сеткой и размещение объектов.

```
self.view = GridView(self.scene, self)
```

После создания сцены формируется центральный интерфейс программы. В верхней части окна размещаются кнопки сохранения и загрузки сцены.

```
for btn in [("сохранить", lambda: self.save_scene()),
            ("загрузить", lambda: self.load_data())]:
```

Основная область окна занята графическим редактором, в котором пользователь размещает объекты дорожной инфраструктуры.

Для обработки внутренних состояний программы используются следующие переменные:

- selected_obj_type – выбранный тип объекта;
- is_auto_mode – состояние автоматического режима;
- failure_mode – состояние режима сбоя;
- current_phase – текущая фаза работы светофоров.

В системе также используются таймеры QTimer. Один таймер отвечает за проверку данных, поступающих с Arduino, второй – за автоматическое переключение фаз светофоров.

```
self.serial_timer = QTimer()
self.serial_timer.timeout.connect(self.check_arduino_input)
self.serial_timer.start(100)
self.traffic_timer = QTimer()
self.traffic_timer.timeout.connect(self.next_traffic_phase)
```

При выборе объекта на сцене автоматически обновляется панель его свойств.

```
self.scene.selectionChanged.connect(self.on_select)
```

2.6. Создание панели инструментов

Каждая панель требует наличие виджета (QWidget) в который будет положен лэйаут со всеми кнопками (QVBoxLayout)

Панель инструментов предназначена для выбора объектов, размещаемых на сцене. Реализация выполнена с использованием QDockWidget, закреплённого в левой части окна программы.

```
toolbar = QDockWidget("Добавить объект")
self.addDockWidget(Qt.LeftDockWidgetArea, toolbar)
```

Внутри панели размещаются кнопки с иконками объектов дорожной инфраструктуры.

```
buttons = [
    ("TLred", "светофор"),
    ("Stop", "знак"),
    ("Pedestrian", "пешеход"),
    ("Zvertical", "зебра"),
    ("Rvertical", "Rvertical"),
    ("Rcrossroads", "Rcrossroads")
]
```

Каждая кнопка содержит изображение объекта и связана с функцией выбора типа объекта.

```
btn.clicked.connect(lambda checked, t=obj_type: self.select_obj_type(t))
```

После выбора тип объекта сохраняется в переменной selected_obj_type, а последующий клик по сцене приводит к созданию нового элемента выбранного типа.

Для отображения изображений используются объекты QIcon, а размер иконок задаётся через QSize.

```
btn.setIcon(QIcon(media_dir + icon))
btn.setIconSize(QSize(35, 35))
```

В результате панель инструментов выполняет роль графического меню редактора сцены и обеспечивает быстрый выбор элементов дорожной инфраструктуры.

2.7. Панель свойств объектов

Для изменения параметров объектов реализована отдельная панель свойств, расположенная в правой части окна программы.

Создание панели выполняется через QDockWidget.

```
self.prop_dock = QDockWidget("Свойства объекта")
```

Содержимое панели формируется динамически функцией `create_prop_form`. Интерфейс зависит от типа выбранного объекта сцены.

Если объект не выбран, пользователю отображается сообщение:

```
form.addRow(QLabel('Объект не выбран'))
```

Для светофоров реализованы кнопки ручного переключения сигналов:

- красный;
- жёлтый;
- зелёный.

```
btn = QPushButton(f"Установить {color}")
```

Дополнительно присутствует кнопка возврата к стандартному режиму работы.

```
btn_std = QPushButton("Стандартный режим")  
btn.clicked.connect(lambda _, c=color: self.set_manual_light_state(item, None))
```

Для пешеходного перехода реализован выбор направления движения.

```
combo.addItems(["vertical", "horizontal"])
```

Для дорожного сегмента `Rvertical` реализован выбор угла поворота.

```
combo.addItems(["0", "90", "180", "270"])
```

Для дорожных знаков реализован выбор типа ограничения:

- Stop;
- Start;
- Block.

```
combo.addItems(["Stop", "Start", "Block"])
```

Также пользователь может выбрать тип транспортного средства через `QRadioButton`.

```
rb = QRadioButton(color)  
form.addRow("", rb)
```

В конце создания панели возвращается данная форма, которая будет применена. Для обновления панели при выборе объекта используем метод `on_select`

```
def on_select(self):  
    items = self.scene.selectedItems()  
    widget = QWidget()  
    widget.setLayout(self.create_prop_form(items[0] if items else None))  
    self.prop_dock.setWidget(widget)
```

После изменения любого параметра вызывается функция `change_prop`, обновляющая свойства объекта и его визуальное отображение.

```
item.props[prop] = value
item.update_visuals()
```

2.8. Панель управления режимами работы системы

В нижней части окна приложения расположена панель управления режимами работы системы. Реализация выполнена через отдельный `QDockWidget`.

```
control_dock = QDockWidget("Панель управления", self)
```

Панель содержит две основные группы:

- автоматический режим;
- режим сбоя.

Автоматический режим отвечает за циклическое переключение сигналов светофоров.

Для управления используются две кнопки:

- запуск цикла;
- остановка цикла.

```
self.btn_auto_start = QPushButton("Старт цикла")
self.btn_auto_stop = QPushButton("Стоп")
```

После запуска режима вызывается функция `start_auto_mode`, которая:

- активирует автоматическую работу;
- запускает таймер переключения фаз;
- записывает событие в журнал.

```
write_data('РЕЖИМ РАБОТЫ', 'автоматический запущен')
```

Переключение сигналов светофора выполняется функцией `next_traffic_phase`.

```
self.update_lights_visuals(
    "green" if self.current_phase == 0 else "red"
)
```

Вторая группа панели отвечает за управление режимом сбоя. Пользователь может вручную активировать или отключить аварийное состояние системы.

```
("Активировать сбой"      self.activate_failure_mode)
("Деактивировать сбой"    self.deactivate_failure_mode)
```

При активации режима сбоя работа автоматического цикла прекращается, а светофоры переводятся в аварийное состояние.

2.9. Автоматическое управление светофорами

В программе реализован механизм автоматического переключения состояний светофоров.

Функция `update_traffic_lights` вызывается таймером через фиксированный интервал времени.

```
self.timer.timeout.connect(self.update_traffic_lights)
```

Алгоритм выполняет:

- обход всех светофоров сцены;
- изменение текущего состояния;
- обновление изображения;
- отправку команды на Arduino.

Изменение состояния выполняется циклически:

- красный;
- жёлтый;
- зелёный.

После изменения состояния вызывается метод обновления визуального представления.

```
obj.update_visuals()
```

Одновременно с этим на микроконтроллер отправляется соответствующая команда через Serial.

Такой подход обеспечивает синхронную работу:

- графического интерфейса;
- логики приложения;
- физического макета светофора.

Для установки ручного режима

```
def set_manual_light_state(self, item, state):  
    if state is None:  
        item.props.pop("ManualState", None)  
    else:  
        item.props["ManualState"] = item.props["State"] = state  
        self.stop_auto_mode()  
        item.update_visuals()
```

2.10. Формирование панели свойств объектов

Для редактирования параметров объектов реализована функция `create_prop_form`. Данная функция динамически создаёт интерфейс свойств в зависимости от типа выбранного объекта сцены.

Основой панели является `QFormLayout`, позволяющий размещать элементы управления в

виде формы.

```
form = QFormLayout()
```

Если объект не выбран, пользователю отображается сообщение:

```
form.addRow(QLabel('Объект не выбран'))
```

При выборе объекта интерфейс автоматически перестраивается под его тип.

Для светофора создаются кнопки ручного переключения состояния:

- красный сигнал;
- жёлтый сигнал;
- зелёный сигнал.

```
for color in ["red", "yellow", "green"]:  
    btn = QPushButton(f"Установить {color}")
```

Каждая кнопка вызывает функцию `set_manual_light_state`, передавая новое состояние светофора.

```
btn.clicked.connect(lambda _, c=color: self.set_manual_light_state(item, c))
```

Дополнительно реализована кнопка возврата к автоматическому режиму.

```
btn_std = QPushButton("Стандартный режим")
```

Для пешеходного перехода реализован выбор направления:

- вертикальное;
- горизонтальное.

```
combo.addItems(["vertical", "horizontal"])
```

Изменение значения автоматически обновляет визуальное отображение объекта.

```
combo.currentTextChanged.connect(  
    lambda val: self.change_prop(item, "Direction", val)  
)
```

2.11. Сохранение и загрузка сцены

Для сохранения конфигурации дорожной сцены реализован механизм сериализации объектов в формат JSON.

Функция `save_scene` выполняет:

- обход объектов сцены;
- сохранение координат;
- сохранение типа объекта;

- сохранение параметров объекта.

```
scene_data = [{
    "x": item.x(),
    "y": item.y(),
    "obj_type": item.obj_type,
    "props": item.props.copy()
}]
```

После формирования структуры данных пользователь выбирает путь сохранения файла через `QFileDialog`.

```
path, _ = QFileDialog.getSaveFileName(
    self,
    'Сохранить сцену',
    "",
    "JSON (*.json)"
)
```

Сохранение выполняется библиотекой `json`.

```
json.dump(scene_data, f)
```

Загрузка сцены реализована функцией `load_data`.

Пользователь выбирает JSON-файл:

```
path, _ = QFileDialog.getOpenFileName(
    self,
    "Load Scene",
    "",
    "JSON (*.json)"
)
```

После загрузки:

- сцена очищается;
- создаются новые экземпляры объектов;
- восстанавливаются свойства объектов.

```
self.scene.addItem(
    SceneObject(
        info["x"],
        info["y"],
        info['obj_type'],
        info.get('props', {})
    )
)
```

Такой подход позволяет:

- сохранять дорожные схемы;
- повторно загружать конфигурации;
- использовать систему как редактор проектов.

2.12. Автоматический режим управления движением

В системе реализован автоматический режим работы светофоров.

При запуске режима:

- активируется циклическое переключение;
- запускается таймер;
- в журнал записывается событие.

```
self.is_auto_mode = True
write_data('РЕЖИМ РАБОТЫ', 'автоматический запущен')
```

Функция `next_traffic_phase` отвечает за смену фаз светофоров.

```
self.update_lights_visuals(
    "green" if self.current_phase == 0 else "red"
)
```

После обновления состояния запускается таймер следующей фазы.

```
self.traffic_timer.start(
    self.phase_intervals[self.current_phase]
)
```

Для обновления светофоров нужно получить список светофоров в сцене

```
def get_all_traffic_lights(self):
    return [item for item in self.scene.items() if isinstance(item, SceneObject) and item.obj_type == "светофор"]

def update_lights_visuals(self, state):
    for light in self.get_all_traffic_lights():
        if "ManualState" not in light.props and light.props.get("State", "red") != state:
            light.props["State"] = state
            light.update_visuals()
```

Фазы циклически переключаются между:

- красным;
- зелёным сигналом.

```
self.current_phase = (self.current_phase + 1) % 2
```

Остановка автоматического режима выполняется функцией `stop_auto_mode`.

```
self.traffic_timer.stop()
```

Дополнительно в журнал записывается информация об остановке режима.

```
write_data(
    'РЕЖИМ РАБОТЫ',
    'автоматический остановлен'
)
```

2.13. Обработка аварийного режима

Для моделирования аварийной ситуации реализован режим сбоя.

Система получает сообщения от Arduino через последовательный интерфейс. Проверка

выполняется таймером каждые 100 мс.

```
self.serial_timer.timeout.connect(
    self.check_arduino_input
)
```

Если от микроконтроллера поступает сообщение 1, активируется аварийный режим.

```
if '1' in msg and not self.failure_mode:
    self.activate_failure_mode()
```

При получении сообщения 2 аварийный режим отключается.

```
elif '2' in msg and self.failure_mode:
    self.deactivate_failure_mode()
```

Во время аварийного режима:

- останавливается автоматический цикл;
- все светофоры переводятся в жёлтый сигнал;
- событие записывается в журнал.

```
self.update_lights_visuals("yellow")
```

При завершении режима:

- состояние системы восстанавливается;
- светофоры переводятся в красный сигнал.

```
self.update_lights_visuals("red")
```

2.14. Ключевые функции в SQLite

Основные функции в SQLite представлены в таблице 4.

Таблица 4 – Функции SQLite

Название	Что делает	Пример
CREATE TABLE	Создать таблицу	CREATE TABLE events (time TEXT)
INSERT INTO	Добавить запись	INSERT INTO events VALUES ('12:00', 'test')
SELECT	Прочитать записи	SELECT * FROM events
UPDATE	Изменить записи	UPDATE events SET time = '13:00' WHERE type='test'
DELETE FROM	Удалить записи	DELETE FROM events WHERE type='test'
DROP TABLE	Удалить таблицу	DROP TABLE events
ORDER BY	Сортировка по номеру строки (новые сверху)	ORDER BY rowid DESC
LIMIT	Только последние 15 записей	LIMIT 15
connect	Подключение к базе данных	sqlite3.connect('events.db')
close()	Закрыть базу данных (обычно после завершения программы)	conn.close()
Cursor=conn.cursor()	Создать курсор (Но conn.execute())	

	создаёт курсор автоматически, поэтому отдельно можно не писать)	
execute('SQL команда', параметры)	Выполнить SQL-запрос	conn.execute('INSERT INTO events VALUES (?, ?, ?)', (time_str, obj_type, event))
commit()	Сохранить изменения в файл	conn.commit()
fetchall() / fetchone()	Получить результаты SELECT-запроса	fetchall() — все строки списком fetchone() — одну строку

2.15. Список использованных источников

Для углублённого изучения тем, рассмотренных в обоих модулях, рекомендуются следующие источники:

Официальная документация

- Qt Documentation — doc.qt.io: полная документация по классам Qt, включая QTimer, QPainter, QWidget;
- PyQt6 Reference Guide — riverbankcomputing.com: документация по Python-привязкам PyQt6;
- Python 3 Documentation — docs.python.org: модули sqlite3, json, os, time;
- Arduino Reference — arduino.cc/reference: справочник по функциям и переменным Arduino.

ГЛАВА 4. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYGAME

Pygame - низкоуровневый игровой движок, где разработчик сам контролирует все события, игровой цикл и отрисовку объектов. В библиотеке нет встроенных функций отрисовки виджетов и компоновщиков, поэтому боковая панель инструментов и панели свойств отрисовываются вручную прямо на основном холсте.

PyQt6 реализует событийно-ориентированную архитектуру. Приложение находится в состоянии ожидания внутри скрытого цикла событий, запускаемого вызовом `app.exec()`. Когда пользователь нажимает кнопку, перемещает мышь или закрывает окно, Qt генерирует соответствующее событие и направляет его целевому виджету. Виджет обрабатывает событие через виртуальный метод — `mousePressEvent`, `paintEvent`, `keyPressEvent`. После завершения обработки управление возвращается в цикл событий, и программа снова переходит в режим ожидания. Разработчик не контролирует, когда и в каком порядке выполняются операции обновления и отрисовки: Qt самостоятельно решает, какой виджет нуждается в перерисовке, и вызывает его `paintEvent`.

Pygame построен на противоположном принципе — активном игровом цикле. Конструкция `while running` выполняется непрерывно, и каждая итерация представляет собой полный кадр. Разработчик явно управляет последовательностью операций: сначала обновляется логика (передвижение объектов, проверка столкновений), затем экран очищается, затем отрисовываются все видимые элементы, затем вызывается `pygame.display.flip()` для отображения кадра. Никакая операция не происходит автоматически — если не вызвать `screen.fill()` в начале кадра, предыдущее изображение останется на экране. Если не проверить события через `pygame.event.get()`, нажатия клавиш будут проигнорированы. Такой подход даёт полный контроль над происходящим, но перекладывает на разработчика всю ответственность за порядок и своевременность операций.

4.1. ШАГ 1 — КОНСТАНТЫ И ВСПОМОГАТЕЛЬНЫЙ ФУНКЦИИ

Перед началом работы с графикой программа выполняет три подготовительных этапа: объявление констант, загрузку изображений и установку связи с микроконтроллером.

4.1.1. Константы и подключение к Arduino UNO

Блок констант задает все фундаментальные параметры, изменения которых автоматически отражается на всей логике и интерфейсе программы.

Блок try/except с serial.Serial устанавливает связь с микроконтроллером. Если же подключение не удалось появится сообщение об этом в терминале среды разработки, но это не мешает использовать программу. Подключение проходит при помощи библиотеки serial на скорости 9600 бод.

```
import pygame as py, serial, os, json, time, sqlite3
from datetime import datetime

py.init()

GRID_SIZE = 35
OBJ_SIZE = GRID_SIZE
WIDTH, HEIGHT = 810, 630
PANEL_Y = 500
font = py.font.SysFont(None, 18)
clock = py.time.Clock()
screen = py.display.set_mode((WIDTH, HEIGHT))
icon = py.image.load("Прил_ОС_КОД 09.02.08-1-2026-M1\Прил_B1_КОД 09.02.08-1-2026-M1\media\TLred.png")
py.display.set_caption("road")
py.display.set_icon(icon)

try:
    arduino = serial.Serial('COM11', 9600, timeout=1)
    time.sleep(2)
except:
    print("arduino none")
    arduino = None
```

4.1.2. Загрузка изображений

Для загрузки изображений используется словарь кортежей imgs и цикл загрузки – это централизованное хранилище всех графических ресурсов.

В отличие от методички PyQt6, где изображения загружались через QPixmap по мере необходимости, здесь все 12 файлов загружаются один раз при старте в словарь imgs. Список кортежей (ключ, имя_файла) сопоставляет логическое имя объекта в коде с реальным файлом. Для каждого формируется путь, проверяется существование файла через os.path.exists, и масштабируется до 35x35 пикселей вызовом pygame.transform.scale. DIR_TO_ROT отвечает за поворот машины во время движения, где координаты:

1. 0, -1 – поворот вверх
2. 0, 1 – поворот вниз

3. -1, 0 – поворот влево
4. 1, 0 – поворот вправо

```

imgs = {}
for name, file in [('road', 'Rvertical.png'), ('crossroad', 'Rcrossroads.png'),
                  ('ped', 'Pedestrian.png'), ('tl_red', 'TLred.png'),
                  ('tl_yellow', 'TLyellow.png'), ('tl_green', 'TLgreen.png'),
                  ('car_red', 'Cvertical.png'), ('car_blue', 'BCvertical.png'),
                  ('start', 'Start.png'), ('stop', 'Stop.png'),
                  ('block', 'Block.png'), ('cross', 'Zhorizontal.png')]:
    path = f"Прил_ОЗ_КОД_09.02.08-1-2026-M1/Прил_В1_КОД_09.02.08-1-2026-M1/media/{file}"
    if os.path.exists(path):
        imgs[name] = py.transform.scale(py.image.load(path), (OBJ_SIZE, OBJ_SIZE))

DIR_TO_ROT = {
    (0, -1): 0, # вверх
    (0, 1): 2, # вниз
    (-1, 0): 1, # влево
    (1, 0): 3, # вправо
}

```

4.2. ШАГ 2 — КЛАСС OBJ

Класс Obj представляет один размещаемый объект на сетке. Его конструктор решает три задачи: привязку к сетке, инициализацию свойств конкретного типа, начальное состояние.

4.2.1. Инициализация и отрисовка объектов класса Obj

Метод `_init_` – описание конструктора. Выравнивание происходит по сетке при помощи формулы $((x - 180) // \text{GRID_SIZE}) * \text{GRID_SIZE} + 180$, чтобы узнать целочисленные координаты для постановки изображений на холст строго по сетке. 180 в данном случае является шириной панели инструментов слева. В этом методе сразу устанавливаются основные характеристики объектов такие как `light` (цвет для светофора), `color` (цвет для машины), `ped_timer` (установка таймера для пешехода), `progress`, `speed`, `stop_timer` (прогресс, скорость передвижения и таймер для машин).

Метод `draw` – отрисовка объекта. Этот метод определяет какое изображение и как его показывать. Для объектов, которые могут вращаться (машина, дорога, пешеходный переход) используется `pygame.transform.rotate (imgs[self.type], self.rot *`

90). Также для машин изображение выбирается по типу и цвету (car_red, car_blue), для светофоров = по текущему состоянию (tl_red, tl_yellow, tl_green)

Метод rect отвечает за получение прямоугольника для проверки коллизии объектов, а именно через collidepoint() определяется попал ли клик мыши по объекту.

Метод возвращает объект через pygame.Rect(self.x, self.y, GRID_SIZE, GRID_SIZE)

```
class Obj:
    def __init__(self, x, y, t, rot=0):
        grid_x = ((x - 180) // GRID_SIZE) * GRID_SIZE + 180
        grid_y = (y // GRID_SIZE) * GRID_SIZE
        self.type = t
        self.x = grid_x
        self.y = grid_y
        self.rot = rot
        self.light = 'red' if t == 'tl' else None
        self.color = 'red' if t == 'car' else None
        self.ped_timer = 0
        self.path = []
        self.progress = 0.0
        self.speed = 0.04
        self.stop_timer = 0

    def draw(self):
        img = None
        if self.type == 'car':
            img = py.transform.rotate(imgs[f"car_{self.color}"], self.rot * 90)
        elif self.type in ['road', 'crossroad', 'cross']:
            img = py.transform.rotate(imgs[self.type], self.rot * 90)
        elif self.type == 'tl':
            img = imgs[f"tl_{self.light}"]
        else:
            img = imgs[self.type]
        if img:
            screen.blit(img, (self.x, self.y))

    def rect(self):
        return py.Rect(self.x, self.y, GRID_SIZE, GRID_SIZE)

    def get_current_cell(self):
        cx = ((self.x + GRID_SIZE//2 - 180) // GRID_SIZE) * GRID_SIZE + 180
        cy = ((self.y + GRID_SIZE//2) // GRID_SIZE) * GRID_SIZE
        return cx, cy
```

4.2.2 Метод update_car()

Метод вызывается каждый кадр только при активном флаге moving. Если путь не задан или progress достиг последнего сегмента, машина добавляется в to_remove. повторное добавление предотвращается проверкой if self in env.to_remove.

Проверка препятствий на текущей клетке (cx, cy) выполняется поиском объектов других типов. При обнаружении знака (стоп) инициируется тридцатикадровая задержка: поле `stop_timer` получает значение 30 и с каждым кадром уменьшается на единицу, блокируя `progress`. При красном сигнале светофора и знаке блок машина также останавливается. Другие препятствия пропускаются без ограничений.

Если же препятствий нет, `progress` увеличивается на значение `speed`. Целая часть определяет текущий сегмент. Координаты машины вычисляются как линейная интерполяция между узлами с коэффициентом t – дробной частью `progress`. Такой подход обеспечивает плавное передвижение без привязки к центру клетки.

```
def update_car(self, env):
    if not self.path or self.progress >= len(self.path) - 1:
        # машина доехала - удаляем
        if self in env.to_remove:
            return
        env.to_remove.append(self)
        return

    # проверка препятствий на ТЕКУЩЕЙ клетке
    cx, cy = self.get_current_cell()
    for obj in env.objs:
        if obj is self:
            continue
        if obj.x == cx and obj.y == cy:
            if obj.type == 'block':
                return
            if obj.type == 'stop':
                if self.stop_timer == 0:
                    self.stop_timer = 30
                if self.stop_timer > 0:
                    self.stop_timer -= 1
                return
            if obj.type == 'tl' and obj.light == 'red':
                return
```

Движение осуществляется следующим образом:

```

# движение
self.progress += self.speed
seg = int(self.progress)
if seg < len(self.path) - 1:
    t = self.progress - seg
    x1, y1 = self.path[seg]
    x2, y2 = self.path[seg + 1]
    self.x = x1 + (x2 - x1) * t
    self.y = y1 + (y2 - y1) * t

    dx = x2 - x1
    dy = y2 - y1
    if dx != 0:
        dir_x = 1 if dx > 0 else -1
        dir_y = 0
    else:
        dir_x = 0
        dir_y = 1 if dy > 0 else -1
    self.rot = DIR_TO_ROT.get((dir_x, dir_y), 0)

```

4.3. ШАГ 3 — КЛАСС EDITOR

Класс Editor – центральный компонент, объединяющий управление данными, отрисовку интерфейса и взаимодействие с Arduino.

4.3.1. Инициализация компонентов класса Editor()

Метод `__init__` - конструктор редактора. Выполняющий начальную настройку всех структур данных и загружает сохраненную сцену. Базовая структура – `self.objs`, хранит в себе список всех объектов, размещенных на сцене. Поле `self.selected` содержит ссылку на выделенный в данный момент объект либо `None`, если выделения нет.

Параметры светофора:

1. Параметры `self.emergency` и `self.auto` управляют режимами светофора. Первый включается по сигналу от Arduino и принудительно устанавливает красный свет, имитируя аварию на линии. Второй активирует автоматическое переключение цвета светофоров по таймеру. Текущий цвет хранится в `self.light`, для прогноза используется `self.next`. Также устанавливается время последнего переключения в `self.last_time` при помощи `pygame.time.get_ticks()`, что позволяет реализовать конечный автомат смены сигналов без блокирующих задержек.

2. Словарь `self.custom_color` отвечает за индивидуальный цвет светофора, который вручную установил пользователь. Ключом служит (x, y) – координаты клетки

светофора, - значением – булево True (индивидуальный цвет установлен) или False (светофор синхронизирован с глобальными).

Параметры симуляции движения:

1. Флаг self.moving включает и выключает движение машин
2. Флаг self.building_path активирует режим прокладки маршрута
3. Список self.path_points хранит в себе точки строящегося пути

Поле self.bd – список записей базы данных. Инициализируется как пустой список. Каждая запись представляет собой кортеж из двух строк: тип объекта, событие. Ограничение в 100 записей предотвращает неконтролируемый рост потребления памяти при длительной работе программы.

Поле self.show_db – флаг видимости таблицы базы данных, по умолчанию false – таблица не показывается.

```
class Editor:
    def __init__(self):
        self.objs = []
        self.selected = None
        self.add_type = None
        self.deleted = False
        self.show_menu = False
        self.mouse_pos = (0, 0)
        self.emergency = False
        self.auto = False
        self.light = 'red'
        self.next = 'green'
        self.last_time = py.time.get_ticks()
        self.custom_color = {}
        self.moving = False
        self.building_path = False
        self.path_points = []
        self.to_remove = [] # список машин на удаление
        self.show_db = False
        self.conn = sqlite3.connect('events.db')
        self.conn.execute('CREATE TABLE IF NOT EXISTS events (time TEXT, type TEXT, event TEXT)')
        self.conn.execute('DELETE FROM events')
        self.conn.commit()
        self.send('red')
        self.load()
        self.add_db_record("система", "программа запущена")
```

4.3.2. Метод add_db_record

Метод принимает 2 строковых параметра: obj_type (тип объекта или источник события), event (описание произошедшего). Временная метка формируется через datetime.now().strftime("%H:%M:%S") и присоединяется к типу объекта, создавая строку

вида «12:34:11 светофор red». Такой формат позволяет видеть хронологию событий без хранения отдельного поля времени.

Готовая запись добавляется в конец списка `self.db`. После добавления проверяется длина списка: при превышении лимита в 100 записей самый старый удаляется вызовом `pop(0)`. Данный подход реализует циклический буфер фиксированной емкости без использования специализированных структур.

Записи в базу данных добавляются при наступлении ключевых событий. Фиксируется запуск программы, регистрируются включение и выключение аварийного режима с типом объекта «авария». Записывается каждое переключение светофоров с указанием текущего цвета. Также сохраняются события загрузки и сохранения карты в json файл. Данный набор точек охватывает все значимые изменения состояния системы.

```
def add_db_record(self, obj_type, event):
    time_str = datetime.now().strftime("%H:%M:%S")
    self.conn.execute('INSERT INTO events VALUES (?, ?, ?)', (time_str, obj_type, event))
    self.conn.commit()
    time_str = datetime.now().strftime("%H:%M:%S")

def get_db_records(self):
    return self.conn.execute('SELECT time, type, event FROM events ORDER BY rowid DESC LIMIT 15').
    fetchall()[::-1]
```

4.3.3. Методы send и check_arduino

Метод `send` отвечает за отправку команд на Arduino. При помощи `encode()` происходит кодирование данных строки в байты.

Метод `check_arduino` читает данные, полученные от Arduino UNO. Метод вызывается каждый кадр и проверяет, есть ли данные в буфере порта. Если получено сообщение “on”, включается режим аварии. Сообщение “off” отключает аварию и возвращает прежний режим работы светофоров. При помощи `decode()` происходит декодирование полученных данных в виде строк и конвертирование их в числовой формат.

```
def send(self, color):
    if arduino and not self.emergency:
        cmd = {'red': 1, 'yellow': 2, 'green': 3}.get(color)
        if cmd is not None:
            arduino.write(f"{cmd}\n".encode())

def check_arduino(self):
    if arduino and arduino.in_waiting:
        try:
            msg = arduino.readline().decode().strip()
            if msg == "on":
                self.set_light('red')
                self.emergency = True
            elif msg == "off":
                self.emergency = False
                self.set_light(self.light)
        except: pass
```

4.3.4. Методы update и set_light – переключение светофоров

Метод update отвечает за автоматическое переключение светофоров с интервалами 5-2-5 секунд. Отсчет времени ведется через `pygame.time.get_ticks()`, который был прописан в `_init_` и в данном методе, для вычисления сколько прошло времени с последнего переключения.

Метод `set_light` – установка цвета светофора. Цвет меняется глобально для всех светофоров сразу, если для конкретного не задан индивидуальный цвет в его панели свойств. Индивидуальные цвета хранятся в словаре `self.custom_color`, где ключом служит кортеж координат светофора.

В методе `update` также добавлен блок выполняющийся только при активном флаге `moving` – который позволяет отслеживать движущиеся машины и удалять их с поля при достижении конца их пути. Список `to_remove` очищается, затем для каждого объекта типа `car` вызывается метод `update`.

Объявление метода класса функции `update`:

```

def update(self):
    if self.emergency:
        return
    if self.auto:
        now = py.time.get_ticks()
        if self.light == 'red' and now - self.last_time > 5000:
            self.set_light('yellow')
            self.next = 'green'
        elif self.light == 'green' and now - self.last_time > 5000:
            self.set_light('yellow')
            self.next = 'red'
        elif self.light == 'yellow' and now - self.last_time > 2000:
            self.set_light(self.next)

    if self.moving:
        self.to_remove = []
        for obj in self.objs:
            if obj.type == 'car':
                obj.update_car(self)
        for obj in self.to_remove:
            if obj in self.objs:
                if self.selected == obj:
                    self.selected = None
                self.objs.remove(obj)

```

Объявление метода класса функции set_light:

```

def set_light(self, color):
    if self.emergency:
        return
    self.light = color
    self.last_time = py.time.get_ticks()
    self.send(color)
    self.add_db_record("светофор", f"переключение на {color}")
    for obj in self.objs:
        if obj.type == 'tl':
            if not self.custom_color.get((obj.x, obj.y), False):
                obj.light = color

```

4.3.5. Метод draw_db_ui

Окно базы данных представляет собой серую панель в нижней части экрана. Данное окно вынесено в отдельный метод и вызывается только при активном флаге show_db.

В верхней части окна размещаются 2 текстовых заголовка: «тип объекта» (слева) и «событие» (с отступом 200 пикселей). Под ними проводится горизонтальная разделительная линия, визуальнo отделяющая шапку таблицы от данных.

Выводятся последние 15 записей базы данных – через `self.db[-15:]`. Для каждой записи тип объекта обрезается до 25 символов (`obj_type[:25]`), событие до 40 символов (`event[:40]`) во избежание выхода текста за границы окна. Вертикальный шаг между строками составляет 18 пикселей. Если записи достигают нижней границы окна, вывод прекращается досрочно.

Кнопка закрытия расположена в правом верхнем углу таблиц с размером 50X20 пикселей.

```
def draw_db_ui(self):
    win_w, win_h = WIDTH - 20, 300
    win_x, win_y = 10, HEIGHT - win_h - 10
    py.draw.rect(screen, (200,200,200), (win_x, win_y, win_w, win_h))
    py.draw.rect(screen, (50,50,50), (win_x, win_y, win_w, win_h), 1)

    screen.blit(font.render("Время", 1, (0,0,0)), (win_x + 10, win_y + 5))
    screen.blit(font.render("Объект", 1, (0,0,0)), (win_x + 80, win_y + 5))
    screen.blit(font.render("Событие", 1, (0,0,0)), (win_x + 200, win_y + 5))
    py.draw.line(screen, (50, 50, 50), (win_x, win_y + 20), (win_x + win_w, win_y + 20), 1)

    y = win_y + 25
    for time_str, obj_type, event in self.get_db_records():
        screen.blit(font.render(time_str, 1, (0,0,0)), (win_x + 10, y))
        screen.blit(font.render(obj_type[:15], 1, (0,0,0)), (win_x + 80, y))
        screen.blit(font.render(event[:30], 1, (0,0,0)), (win_x + 200, y))
        y += 18
    if y > win_y + win_h - 20:
        break

    close_rect = py.Rect(win_x + win_w - 60, win_y + 5, 50, 20)
    py.draw.rect(screen, (150, 150, 150), close_rect)
    py.draw.rect(screen, (0, 0, 0), close_rect, 1)
    screen.blit(font.render("x", 1, (0,0,0)), (close_rect.x + 20, close_rect.y + 3))

    mx, my = py.mouse.get_pos()
    if py.mouse.get_pressed()[0]:
        if close_rect.collidepoint(mx, my):
            self.show_db = False
```

4.3.6. Метод draw_ui – отрисовка пользовательского интерфейса

В Pygame интерфейс рисуется процедурно: кнопки, текст, панели свойств – все создается вызовами `py.draw.rect` и `screen.blit` внутри метода. Это самый объемный метод в программе.

```

def draw_ui(self):
    btns = [('add', 'добавить'), ('delete', f"удалить {'on' if self.deleted else 'off'}"),
            ('load', 'загрузить'), ('save', 'сохранить'), ('red', 'красный'),
            ('yellow', 'желтый'), ('green', 'зеленый'),
            ('auto', f"авто {'on' if self.auto else 'off'}"),
            ('move', f"движение {'on' if self.moving else 'off'}"), ('db', 'таблица')]

    for i, (cmd, name) in enumerate(btns):
        rect = py.Rect(10, 10 + i*25, 160, 20)
        color = (200, 200, 200)
        py.draw.rect(screen, color, rect)
        py.draw.rect(screen, (0,0,0), rect, 1)
        screen.blit(font.render(name, 1, (0,0,0)), (rect.x + 5, rect.y + 3))

    items = [('road', 'дорога'), ('crossroad', 'перекресток'),
            ('car', 'машина'), ('tl', 'светофор'),
            ('ped', 'пешеход'), ('cross', 'переход'),
            ('stop', 'стоп'), ('start', 'старт'), ('block', 'блок')]

    if self.show_menu:
        for i, (cmd, title) in enumerate(items):
            rect = py.Rect(10, 10 + (len(btns) + i)*25, 160, 20)
            color = (220, 220, 220)
            py.draw.rect(screen, color, rect)
            py.draw.rect(screen, (0,0,0), rect, 1)
            screen.blit(font.render(title, 1, (0,0,0)), (rect.x + 5, rect.y + 3))

```

Структура: сначала рисуются кнопки основного меню, затем — выпадающее меню добавления объектов, потом — панели свойств для пешеходов, светофоров и знаков. Все панели отрисовываются внизу панели инструментов на высоте 500 пикселей, за что отвечает переменная `PANEL_Y`.

Также добавлены текстовые оповещение, если включён режим постройки путей для машин и оповещение об аварии внизу экрана.

Условие if-for:

```

if self.selected and self.selected.type == 'ped':
    panel = py.Rect(10, PANEL_Y, 160, 80)
    py.draw.rect(screen, (220, 220, 220), panel)
    py.draw.rect(screen, (0,0,0), panel, 1)

    y = PANEL_Y + 5
    screen.blit(font.render(f"таймер: {self.selected.ped_timer} сек", 1, (0,0,0)), (20, y))
    y += 15
    for label, cmd in [("+1", 1), ("-1", -1)]:
        rect = py.Rect(20, y, 50, 20)
        py.draw.rect(screen, (220, 220, 220), rect)
        py.draw.rect(screen, (0,0,0), rect, 1)
        screen.blit(font.render(label, 1, (0,0,0)), (rect.x + 10, rect.y + 3))
        y += 25

```

Условие if-for:

```

if self.selected and self.selected.type == 't1':
    panel = py.Rect(10, PANEL_Y, 160, 100)
    py.draw.rect(screen, (220, 220, 220), panel)
    py.draw.rect(screen, (0,0,0), panel, 1)

    y = PANEL_Y + 5
    screen.blit(font.render("цвет: ", 1, (0,0,0)), (20, y))
    y += 15
    colors = [('red', 'красный'), ('green', 'зеленый'), ('standart', 'стандарт')]
    for col_code, col_name in colors:
        rect = py.Rect(20, y, 120, 20)
        py.draw.rect(screen, (230, 230, 230), rect)
        py.draw.rect(screen, (0,0,0), rect, 1)
        screen.blit(font.render(f"{col_name}", 1, (0,0,0)), (rect.x + 10, rect.y + 3))
        y += 25

```

Условие if-for:

```

if self.selected and self.selected.type in ['stop', 'start', 'block']:
    panel = py.Rect(10, PANEL_Y, 160, 100)
    py.draw.rect(screen, (220, 220, 220), panel)
    py.draw.rect(screen, (0,0,0), panel, 1)

    y = PANEL_Y + 5
    screen.blit(font.render("цвет: ", 1, (0,0,0)), (20, y))
    y += 15
    signs = [('stop', 'стоп'), ('start', 'старт'), ('block', 'блок')]
    for sign_code, sign_name in signs:
        rect = py.Rect(20, y, 100, 20)
        py.draw.rect(screen, (230, 230, 230), rect)
        py.draw.rect(screen, (0,0,0), rect, 1)
        screen.blit(font.render(f"{sign_name}", 1, (0,0,0)), (rect.x + 10, rect.y + 3))
        y += 25

```

Условие if-for:

```

if self.building_path and self.selected and self.selected.type == 'car':
    info = font.render("строй путь: кликай по соседним клеткам | ENTER - готово | ESC - отмена", 1, (0,0,0))
    screen.blit(info, (180, HEIGHT-25))
    if self.emergency:
        screen.blit(font.render("АВАРИЯ", 1, (255,0,0)), (500, 610))

coord_text = f"x: {self.mouse_pos[0]} y:{self.mouse_pos[1]}"
screen.blit(font.render(coord_text, 1, (0,0,0)), (self.mouse_pos[0], self.mouse_pos[1] + 20))

```

4.3.7. Методы save, load — сохранение и загрузка сцены

Сохранение и загрузка сцены происходит в json файл.

Структура данных при сохранении – список словарей с ключами x, y, t (тип объекта), rot, light, color, ped_time.

При загрузке для каждого словаря создается объект obj и восстанавливаются все его свойства из json файла.

```

def save(self):
    data = [{'x': o.x, 'y': o.y, 't': o.type, 'rot': o.rot, 'light': o.light,
            'color': o.color, 'ped_timer': o.ped_timer, 'path': o.path}
            for o in self.objs]
    with open('scene.json', 'w') as f:
        json.dump(data, f)
    self.add_db_record("файл", "сохранение карты")

def load(self):
    try:
        with open('scene.json', 'r') as f:
            self.objs = []
            data = json.load(f)
            for d in data:
                obj = Obj(d['x'], d['y'], d['t'], d['rot'])
                if 'light' in d: obj.light = d['light']
                if 'color' in d: obj.color = d['color']
                if 'ped_timer' in d: obj.ped_timer = d['ped_timer']
                obj.path = d.get('path', [])
                self.objs.append(obj)
            self.add_db_record("файл", "загрузка карты")
    except: pass

```

4.3.8. Вспомогательные функции размещения объектов на холсте

Функция `get_cell_type` определяет тип объекта в указанной клетке. Просматривает список в обратном порядке (`reversed`), потому что последний добавленный объект верхнего слоя должен перекрывать нижние. Возвращает строку типа `None`, если клетка пуста.

Функции `is_nearby_crossroad` и `is_nearby_cross` проверяют соседние клетки (сверху, снизу, слева, справа) на наличие перекрестка или пешеходного перехода. Смещение всегда равно размеру клетки (`grid_size = 35`). Эти функции – ключевой элемент правил размещения.

```

def get_cell_type(x, y, objs):
    for obj in reversed(objs):
        if obj.x == x and obj.y == y:
            return obj.type
    return None

def is_nearby_crossroad(x, y, objs):
    for dx, dy in [(0, GRID_SIZE), (0, -GRID_SIZE), (GRID_SIZE, 0), (-GRID_SIZE, 0)]:
        if get_cell_type(x + dx, y + dy, objs) == 'crossroad':
            return True
    return False

def is_nearby_cross(x, y, objs):
    for dx, dy in [(0, GRID_SIZE), (0, -GRID_SIZE), (GRID_SIZE, 0), (-GRID_SIZE, 0)]:
        if get_cell_type(x + dx, y + dy, objs) == 'cross':
            return True
    return False

```

Функция `can_place` реализует правила размещения объектов:

- Светофор можно ставить только на дорогу и только рядом с перекрёстком
- Пешеход – на пустую клетку рядом с пешеходным переходом
- Машины и знаки – на дороге или перекрестке
- Дорога и перекресток – без ограничений

Возвращает тип клетки и можно ли располагать ее в клетке

```
def can_place(obj_type, grid_x, grid_y, objs):
    cell_type = get_cell_type(grid_x, grid_y, objs)
    if obj_type == 'tl':
        return cell_type in ['road', 'crossroad'] and is_nearby_crossroad

    elif obj_type == 'ped':
        return cell_type is None and is_nearby_cross(grid_x, grid_y, objs)

    elif obj_type in ['car', 'stop', 'block', 'start', 'cross']:
        return cell_type in ['road', 'crossroad']

    else: return True
```

Функция `is_adjacent` – проверка соседства двух клеток.

Две клетки считаются соседними, если они расположены на одной строке или на одном столбце и расстояние между ними составляет один шаг. Условие реализовано через проверку координат: либо модуль разности по X равен размеру клетки при равных Y, либо модуль разности по Y равен размеру клетки при равных X. функция гарантирует, что маршрут строится только по горизонтали или вертикали, исключая диагональное построение.

```
def is_adjacent(p1, p2):
    return (abs(p1[0] - p2[0]) == GRID_SIZE and p1[1] == p2[1]) or \
           (abs(p1[1] - p2[1]) == GRID_SIZE and p1[0] == p2[0])
```

4.4. ШАГ 4 — ОБРАБОТКА СОБЫТИЙ И ИГРОВОЙ ЦИКЛ

Главный игровой цикл представляет собой центральный механизм любого Pygame-приложения. Конструкция `while running` связывает логику, отрисовку и обработку пользовательского ввода в единый непрерывный процесс.

Вызов `ed.update()` – обновление логики редактора. Данный вызов выполняется первым до начала отрисовки, для старта отсчета времени работы программы и отслеживания состояния светофоров.

Отрисовка сетки происходит в циклах `for x in range(...)` и `for y in range(...)` для удобства расположения объектов на сцене. Сетка рисуется поверх фона, но до вызова размещения объектов.

Отрисовка объектов – цикл `for obj in ed.objs`. все объекты сцены последовательно вызывают свой метод `draw()` из класса `Obj`, определяя нужное изображение по типу объекта. Если объект является выделенным (`ed.selected`), вокруг него отрисовывается желтая рамка толщиной в 1 пиксель через `pygame.draw.rect(screen, (255, 255, 0), obj.rect(), 1)`.

После отрисовки объектов вызывается метод `ed.draw_ui` для рисовки всего интерфейса и метод `ed.show_db` отрисовки таблицы с базой данных.

```
ed = Editor()
running = True
while running:
    ed.update()
    screen.fill((240, 240, 240))

    for x in range(180, WIDTH, GRID_SIZE):
        for y in range(0, HEIGHT, GRID_SIZE):
            py.draw.rect(screen, (220, 220, 220), (x, y, GRID_SIZE, GRID_SIZE), 1)

    for obj in ed.objs:
        obj.draw()
        if ed.selected == obj:
            py.draw.rect(screen, (255, 255, 0), obj.rect(), 1)

    ed.draw_ui()
    if ed.building_path and len(ed.path_points) > 1:
        for i in range(len(ed.path_points)-1):
            x1, y1 = ed.path_points[i]
            x2, y2 = ed.path_points[i+1]
            py.draw.line(screen, (0,0,255), (x1+GRID_SIZE//2, y1+GRID_SIZE//2), (x2+GRID_SIZE//2,
            y2+GRID_SIZE//2), 3)
    if ed.show_db:
        ed.draw_db_ui()
```

4.4.1. Обработка событий

Блок `for event in pygame.event.get()` извлекает все события, накопившиеся с предыдущего кадра, и направляет их в соответствующие обработчики.

Ниже приведены главные события для управления программой

Основные события в Pygame

Событие	Описание
Pygame.QUIT	Завершение работы программы при нажатии на кнопку закрытия окна
Pygame.KEYDOWN	Обработка клавиатурных сокращений

Pygame.MOUSEBUTTONDOWN	Обработка клика мыши
Pygame.MOUSEMOTION	Отслеживание позиции мыши
Pygame.MOUSEBUTTONUP	Обработка отпускания кнопки мыши
Pygame.MOUSEWHEEL	Обработка кручения колесика мыши

Клавиша `escape` выполняет сброс всех режимов: скрывает режим добавления объектов, отключает режим удаления, снимает текущее выделение.

Клавиша `r` предназначена для поворота некоторых объектов (машины, дорога, переход) на 90 градусов по формуле $(rot + 1) \% 4$.

Клавиша `c` предназначена для смены цвета (красный или синий) для машин.

Клавиша `p` предназначена для запуска и завершения режима построения пути. Сперва необходимо выделить нужную машину кликом мыши по ней и нажать «`p`», начальная точка маршрута совпадает с позицией машины. Если режим уже активен, повторное нажатие завершает его без сохранения маршрута. Чтобы сохранить маршрут необходимо нажать клавишу `enter`, если длина пути состоит из более 2х точек. Список присваивается полю `path` выделенной машины, а `progress` сбрасывается до 0.0. режим построения отключается, список точек очищается. Также для сброса режима построения можно использовать клавишу `escape` – очищаются `building_path`, `path_point`, сбрасывается режим построения.

```

for event in py.event.get():
    if event.type == py.QUIT:
        running = False

    elif event.type == py.KEYDOWN:
        if event.key == py.K_ESCAPE:
            ed.show_menu = False
            ed.add_type = None
            ed.deleted = False
            ed.selected = None
            ed.building_path = False
            ed.path_points = []
            ed.show_db = False
        elif event.key == py.K_r and ed.selected:
            ed.selected.rot = (ed.selected.rot + 1) % 4
        elif event.key == py.K_c and ed.selected and ed.selected.type == 'car':
            ed.selected.color = 'blue' if ed.selected.color == 'red' else 'red'
        elif event.key == py.K_p:
            if ed.selected and ed.selected.type == 'car':

```

Условия:

```
if not ed.building_path:
    ed.building_path = True
    ed.path_points = [(ed.selected.x, ed.selected.y)]
else:
    ed.building_path = False
    ed.selected.path = ed.path_points
    ed.selected.progress = 0.0
    ed.path_points = []
elif event.key == py.K_RETURN and ed.building_path:
    if len(ed.path_points) >= 2:
        ed.selected.path = ed.path_points
        ed.selected.progress = 0.0
    ed.building_path = False
    ed.path_points = []
```

4.4.2. Событие MOUSEBUTTONDOWN

Событие `pygame.MOUSEBUTTONDOWN` – наиболее сложный блок обработки. Сначала проверяется нажатие мыши по закрывающему крестику таблицы с базой данных, при помощи метода `collidepoint()` сравниваются координаты мыши и координаты кнопки закрытия.

Далее проверяются клики мыши для построения пути. При активном `building_path` координаты клика преобразуются в сеточные. Размещение разрешено только на клетках, где установлены дорога или перекресток, при этом другие объекты такие как другие машины, знаки, светофоры или пешеходный переход при построении не учитываются. Добавляемая точка обязана быть соседней с последней точкой маршрута. Повторное добавление одной и той же клетки запрещено сравнением с последним элементом списка.

После проверяются клики по панелям свойств, если выделен объект соответствующего типа.

- Для пешеходов созданы кнопки «+1» и «-1» таймера
- Для светофора - выбор индивидуального цвета (красный и зеленый) и кнопка, которая возвращает состояние выделенного светофора в стандартное
- Для знаков (стоп, старт, блок) - кнопки переключения между типами

Клики мыши проверяются при помощи простого сравнения координат мыши и координат самих кнопок. Каждая кнопка имеет высоту в 20 пикселей и длину 60-120 пикселей с отступом между в 5 пикселей.

Условия:

```

elif event.type == py.MOUSEBUTTONDOWN:
    x, y = event.pos
    if ed.show_db:
        win_w, win_h = WIDTH - 20, 300
        win_x, win_y = 10, HEIGHT - win_h - 10
        close_rect = py.Rect(win_x + win_w - 60, win_y + 5, 50, 20)
        if close_rect.collidepoint(x, y):
            ed.show_db = False
            continue

    if ed.building_path:
        grid_x = ((x - 180) // GRID_SIZE) * GRID_SIZE + 180
        grid_y = (y // GRID_SIZE) * GRID_SIZE
        new_point = (grid_x, grid_y)

        if get_cell_type(grid_x, grid_y, ed.objs) in ['road', 'crossroad', 'cross', 'tl', 'car',
        'stop', 'start', 'block']:
            if ed.path_points and not is_adjacent(ed.path_points[-1], new_point):
                continue
            if not ed.path_points or ed.path_points[-1] != new_point:
                ed.path_points.append(new_point)
            continue

```

Условия:

```

if ed.selected and ed.selected.type == 'ped':
    if 10 <= x <= 70 and PANEL_Y <= y <= HEIGHT:
        if PANEL_Y + 20 <= y <= PANEL_Y + 40:
            ed.selected.ped_timer += 1
        elif PANEL_Y + 45 <= y <= PANEL_Y + 65:
            ed.selected.ped_timer = max(0, ed.selected.ped_timer - 1)
        continue

    elif ed.selected and ed.selected.type == 'tl':
        if 10 <= x <= 130 and PANEL_Y <= y <= HEIGHT:
            if PANEL_Y + 20 <= y <= PANEL_Y + 40:
                ed.selected.light = 'red'
                ed.custom_color[(ed.selected.x, ed.selected.y)] = True
            elif PANEL_Y + 45 <= y <= PANEL_Y + 65:
                ed.selected.light = 'green'
                ed.custom_color[(ed.selected.x, ed.selected.y)] = True
            elif PANEL_Y + 70 <= y <= PANEL_Y + 90:
                ed.selected.light = ed.light
                ed.custom_color[(ed.selected.x, ed.selected.y)] = False
            continue

```

Условия:

```

elif ed.selected and ed.selected.type in ['stop', 'start', 'block']:
    if 10 <= x <= 110 and PANEL_Y <= y <= HEIGHT:
        if PANEL_Y + 20 <= y <= PANEL_Y + 40:
            ed.selected.type = 'stop'
        elif PANEL_Y + 45 <= y <= PANEL_Y + 65:
            ed.selected.type = 'start'
        elif PANEL_Y + 70 <= y <= PANEL_Y + 90:
            ed.selected.type = 'block'
        continue

```

Затем следует обработка клика по основному меню ($x < 180$). Индекс кнопки определяется целочисленным делением: $(y - 10) // 25$. Восемь кнопок соответствует командам: 'add' (показать меню для добавления объекта), 'delete' (включить/выключить режим удаления объектов), 'load' (загрузить сохраненную сцену), 'save' (сохранить текущую сцену), 'red', 'yellow', 'green' (ручное переключение всех светофоров), 'auto' (включение режима автопереключение цвета светофоров), 'move' (включение движения машин по путям), 'db' (показать таблицу с базой данных).

Если меню добавление объектов открыто, дополнительные индексы проверяются после девяти типов добавляемых объектов: дорога, перекресток, машина, светофор, пешеход, переход, стоп, старт, блок. Выбор устанавливает `ed.add_type` в соответствующий строковый идентификатор и закрывает меню выбора объекта.

```

if x < 180:
    idx = (y - 10) // 25
    btns = ['add', 'delete', 'load', 'save', 'red', 'yellow', 'green', 'auto', 'move', 'db']
    if idx < len(btns):
        cmd = btns[idx]
        if cmd == 'add':
            ed.show_menu = not ed.show_menu
            ed.deleted = False
        elif cmd == 'delete':
            ed.deleted = not ed.deleted
            ed.show_menu = False
            ed.add_type = None
        elif cmd == 'load':
            ed.load()
            ed.add_type = None
        elif cmd == 'save':
            ed.save()
        elif cmd in ['red', 'yellow', 'green']:
            ed.set_light(cmd)
        elif cmd == 'auto':
            ed.auto = not ed.auto
        elif cmd == 'move':
            ed.moving = not ed.moving
        elif cmd == 'db':
            ed.show_db = not ed.show_db

```

Обработка клика по сетке ($x > 180$). В режиме удаления ищется объект, в чей прямоугольник `rect()` попали координаты при помощи `collidepoint()`, и удаляется из списка.

В режиме добавления объектов вычисляются сеточные координаты, проверяется возможность размещения через функцию `can_place`, и при успехе создаётся новый экземпляр `Obj()`.

```

|         if ed.show_menu:
|             idx_items = (y - 10 - len(btns)*25) // 25
|             items = ['road', 'crossroad', 'car', 'tl', 'ped', 'cross', 'stop', 'start', 'block']
|             if 0 <= idx_items < len(items):
|                 ed.add_type = items[idx_items]
|                 ed.show_menu = False

```

В обычном режиме выполняется поиск объекта под курсором и установка его как выделенного. Используется `reversed()` для списка объектов, чтобы последний поставленный объект на этой клетке стал выделенным.

```

|     else:
|         if ed.deleted:
|             for obj in reversed(ed.objs):
|                 if obj.rect().collidepoint(x, y):
|                     ed.objs.remove(obj)
|                     ed.add_db_record("объект", f"удален {obj.type}")
|                     if ed.selected == obj:
|                         ed.selected = None
|                     break
|
|             elif ed.add_type:
|                 grid_x = ((x - 180) // GRID_SIZE) * GRID_SIZE + 180
|                 grid_y = (y // GRID_SIZE) * GRID_SIZE
|                 if can_place(ed.add_type, grid_x, grid_y, ed.objs):
|                     obj = Obj(x, y, ed.add_type)
|                     ed.objs.append(obj)
|                     ed.add_db_record("объект", f"добавлен {obj.type}")
|                     ed.selected = obj
|
|             else:
|                 ed.selected = None
|                 for obj in reversed(ed.objs):
|                     if obj.rect().collidepoint(x, y):
|                         ed.selected = obj
|                         break

```

Событие `pygame.MOUSEMOTION` – отслеживание позиции мыши. Сохраняет текущие координаты курсора в `ed.mouse_pos` для их отображения в строке состояния под курсором.

```
elif event.type == py.MOUSEMOTION:  
    ed.mouse_pos = event.pos
```

4.4.3. Завершающие события

Вызов `ed.arduino_check()` – проверка данных, получаемые с микроконтроллера.

Вызов `Pygame.display.flip()` – обновление экрана. Без `flip()` изображение на экране не изменится – этот вызов является обязательным завершением каждого кадра.

Вызов `clock.tick(60)` – ограничение частоты кадров в 60 fps. Приостанавливает выполнение цикла на время. Возвращаемое значение (количество миллисекунд, прошедших с предыдущего вызова) в данной программе не используется, но доступно для расчетов физики или анимации.

```
ed.check_arduino()  
py.display.flip()  
clock.tick(60)  
  
if arduino:  
    arduino.close()  
ed.conn.close()  
py.quit()
```

4.5. ВЗАИМОДЕЙСТВИЕ С ARDUINO UNO

Связка Python-приложения с микроконтроллером Arduino реализует двустороннее взаимодействие: компьютер управляет светодиодами светофора, а микроконтроллер может сигнализировать о нажатии аварийной кнопки. Прошивка Arduino принимает числовые команды и отправляет текстовые сообщения, формируя простой, но надёжный протокол обмена.

Взаимодействие между Python-приложением и Arduino построено на асимметричном протоколе. Компьютер отправляет одnobайтовые команды (цифры 1, 2, 3, закодированные в ASCII), завершаемые символом перевода строки. Микроконтроллер отправляет текстовые сообщения "on" и "off", также завершаемые переводом строки. Согласованность форматов обеспечивается парой методов: `arduino.write(f'{cmd}\n'.encode())` на стороне Python и `Serial.parseInt() / Serial.println()` на стороне Arduino.

Двухсекундная пауза `time.sleep(2)` после установки соединения на стороне Python синхронизирована с поведением Arduino: при открытии последовательного порта микроконтроллер автоматически перезагружается, и это время необходимо для

завершения загрузчика и запуска `setup()`. Попытка отправить данные до истечения этой паузы привела бы к их потере.

4.5.1. Константы и `setup()`

Пины микроконтроллера распределены на две функциональные группы. Выходные каналы `redPin = 7`, `yellowPin = 8`, `greenPin = 9` управляют соответствующими светодиодами светофора. Входные каналы `emergencyBtn = 11` и `resetBtn = 12` подключены к кнопкам с подтяжкой к питанию — при нажатии на пине устанавливается низкий уровень. Режим `INPUT_PULLUP` задействует встроенные резисторы микроконтроллера, избавляя от необходимости устанавливать внешние.

Глобальная переменная `emergency` хранит текущее состояние аварийного режима. При её активации управление светодиодами перехватывается локально, и команды с компьютера игнорируются.

Настройка начинается с конфигурации всех четырёх пинов: выходные каналы устанавливаются в режим `OUTPUT`, входные — в `INPUT_PULLUP`. Последовательный порт открывается на скорости 9600 бод, что соответствует настройке `serial.Serial('COM11', 9600)` на стороне Python. Скорость выбрана стандартной для Arduino, обеспечивая достаточную пропускную способность при минимальных помехах.

Немедленно после инициализации вызывается `setLight(1)` — светофор принудительно переводится в режим красного сигнала. Данное решение гарантирует безопасное начальное состояние независимо от того, запущено ли Python-приложение.

```
const int redPin = 7;
const int yellowPin = 8;
const int greenPin = 9;

const int emergencyBtn = 11;
const int resetBtn = 12;
bool emergency = false;

void setup() {
    pinMode(redPin, OUTPUT);
    pinMode(yellowPin, OUTPUT);
    pinMode(greenPin, OUTPUT);
    pinMode(emergencyBtn, INPUT_PULLUP);
    pinMode(resetBtn, INPUT_PULLUP);

    Serial.begin(9600);
    setLight(1);
}
```

4.5.2. Основной цикл loop()

Цикл разделён на две логические ветви, выбор между которыми определяется флагом emergency.

Обнаружение нажатий кнопок. Для защиты отдребезга контактов реализован программный детектор фронта. Статические переменные lastEm и lastReset хранят предыдущее состояние кнопок. Нажатие фиксируется только при переходе из высокого уровня в низкий: условие emPressed && !lastEm истинно ровно один цикл на каждое нажатие. Без этого механизма за одно нажатие сгенерировались бы десятки сообщений.

При обнаружении нажатия аварийной кнопки флаг emergency устанавливается в true и отправляется сообщение "on". При нажатии кнопки сброса флаг сбрасывается и передаётся "off". Эти строковые сообщения принимаются на стороне Python методом check_arduino() через arduino.readline().decode().strip().

```
void loop() {  
    static int lastEm = 1;  
    static int lastReset = 1;  
    bool emPressed = (digitalRead(emergencyBtn) == 0);  
    bool resetPressed = (digitalRead(resetBtn) == 0);  
  
    if (emPressed && !lastEm) {  
        emergency = true;  
        Serial.println("on");  
    }  
  
    if (resetPressed && !lastReset) {  
        emergency = false;  
        Serial.println("off");  
    }  
  
    lastEm = emPressed;  
    lastReset = resetPressed;  
}
```

Аварийный режим. Когда emergency активен, светодиоды управляются локально, игнорируя последовательный порт. Красный светодиод включается на 500 миллисекунд, затем выключается на 500 миллисекунд — формируется мигающий сигнал, привлекающий внимание. Жёлтый и зелёный светодиоды в аварийном режиме принудительно погашены. Одновременно с миганием выполняется задержка delay(500), что делает этот режим блокирующим — сброс аварии произойдёт только при следующей итерации цикла.

```
if (emergency) {  
    digitalWrite(redPin, 1);  
    digitalWrite(yellowPin, 0);  
    digitalWrite(greenPin, 0);  
    delay(500);  
    digitalWrite(yellowPin, 0);  
    delay(500);  
}  
else {  
    if (Serial.available() > 0) {  
        int cmd = Serial.parseInt();  
        if (cmd >= 1 && cmd <= 3) {  
            setLight(cmd);  
        }  
    }  
    delay(100);  
}  
}
```

Рабочий режим. При отсутствии аварии микроконтроллер проверяет наличие данных в буфере последовательного порта вызовом `Serial.available()`. Если байты доступны, `Serial.parseInt()` извлекает целое число из входного потока. Данный метод автоматически отбрасывает нецифровые символы и ожидает поступления всех цифр в течение таймаута. Если число находится в допустимом диапазоне от 1 до 3, вызывается `setLight()`. После обработки команды выполняется задержка `delay(100)` для предотвращения перегрузки порта.

4.5.3. Функция setLight()

Функция реализует безопасную смену состояния светофора. Сначала все три светодиода принудительно выключаются — это гарантирует, что два сигнала не загорятся одновременно при ошибке передачи команды. Затем включается один светодиод в зависимости от аргумента `cmd`: 1 включает красный, 2 — жёлтый, 3 — зелёный.

Числовые коды соответствуют словарию `{'red': 1, 'yellow': 2, 'green': 3}` в методе `send()` Python-программы. Согласованность этих кодов критична: расхождение приведёт к неверной индикации без каких-либо сообщений об ошибке.

```
void setLight(int cmd) {  
    digitalWrite(redPin, 0);  
    digitalWrite(yellowPin, 0);  
    digitalWrite(greenPin, 0);  
  
    if (cmd == 1){  
        digitalWrite(redPin, 1);  
    }  
  
    else if (cmd == 2){  
        digitalWrite(yellowPin, 1);  
    }  
  
    else if (cmd == 3){  
        digitalWrite(greenPin, 1);  
    }  
}
```

4.6. ТИПИЧНЫЕ ОШИБКИ ПРОГРАММЫ НА PYGAME

1. Изображения не загружаются

Если в консоли выводятся сообщения "no ..." — проверьте, что папка media находится в рабочем каталоге. В отличие от методички, где используется QPixmap с проверкой isNull(), здесь проверка через os.path.exists. Ошибка возникает, если программа запущена не из папки проекта.

2. Объект не ставится на клетку

Проверьте, что координаты мыши правильно преобразуются в сеточные. Формула $((x - 180) // \text{GRID_SIZE}) * \text{GRID_SIZE} + 180$ требует, чтобы x был больше 180. Клик по панели управления не должен размещать объекты.

3. Кнопки не нажимаются

В методичке используется QPushButton с автоматической обработкой кликов. Здесь координаты кнопок вычисляются вручную. Если изменить размер шрифта или отступы, все формулы $10 + i * 25$ придётся пересчитывать — это плата за отсутствие компоновщика.

4. Поворот не применяется

Клавиша R работает только при выбранном объекте (ed.selected не None). Если выделения нет, нажатие игнорируется. Это поведение отличается от методички, где кнопка «Повернуть» создаётся динамически в панели свойств и всегда связана с объектом.

4.7. КРАТКИЙ СПРАВОЧНИК PYGAME

Основные модули в pygame представлены ниже.

Основные модули

Модуль	Описание
pygame.display	Управление окном и обновление экрана
pygame.draw	Рисование геометрических фигур
pygame.image	Загрузка и сохранение изображений
pygame.transform	Масштабирование и поворот
pygame.event	Обработка событий
pygame.time	Часы и задержки
pygame.font	Работа с текстом

Основные функции в pygame представлены ниже.

Функции pygame

Название	Описание
pygame.display.set_mode((w, h))	Создать окно
pygame.display.flip()	Обновить экран
pygame.image.load(path)	Загрузить изображение
pygame.transform.scale(img, (w, h))	Изменить размер
pygame.transform.rotate(img, angle)	Повернуть изображение
pygame.draw.rect(surface, color, rect, width)	Нарисовать прямоугольник
pygame.Rect(x, y, w, h)	Создать прямоугольник для коллизий
rect.collidepoint(x, y)	Проверить попадание точки
font.render(text, antialias, color)	Создать поверхность с текстом
screen.blit(surface, (x, y))	Нарисовать одну поверхность на другой
pygame.time.Clock().tick(fps)	Ограничить частоту кадров
pygame.time.get_ticks()	Миллисекунды с момента запуска

4.8. КЛЮЧЕВЫЕ ФУНКЦИИ В SQLITE

Основные функции в SQLite представлены ниже.

Функции SQLite

Название	Что делает	Пример
CREATE TABLE	Создать таблицу	CREATE TABLE events (time TEXT)
INSERT INTO	Добавить запись	INSERT INTO events VALUES ('12:00', 'test')
SELECT	Прочитать записи	SELECT * FROM events
UPDATE	Изменить записи	UPDATE events SET time = '13:00' WHERE type='test'
DELETE FROM	Удалить записи	DELETE FROM events WHERE type='test'
DROP TABLE	Удалить таблицу	DROP TABLE events

ORDER BY	Сортировка по номеру строки (новые сверху)	ORDER BY rowid DESC
LIMIT	Только последние 15 записей	LIMIT 15
connect	Подключение к базе данных	sqlite3.connect('events.db')
close()	Закрыть базу данных (обычно после завершения программы)	conn.close()
Cursor=conn.cursor()	Создать курсор (Но conn.execute() создаёт курсор автоматически, поэтому отдельно можно не писать)	
execute('SQL команда', параметры)	Выполнить SQL-запрос	conn.execute('INSERT INTO events VALUES (?, ?, ?)', (time_str, obj_type, event))
commit()	Сохранить изменения в файл	conn.commit()
fetchall() / fetchone()	Получить результаты SELECT-запроса	fetchall() — все строки списком fetchone() — одну строку

ГЛАВА 5. РЕАЛИЗАЦИЯ ПРОГРАММЫ НА PYSIDE6

Введение в PySide6

PySide6 — это официальный набор привязок Python к фреймворку Qt6, разрабатываемый и поддерживаемый непосредственно компанией The Qt Company. Он позволяет создавать кроссплатформенные графические интерфейсы (GUI) для десктопных и мобильных приложений, объединяя простоту и динамичность Python с мощностью и надежностью промышленного фреймворка Qt, написанного на C++

Многие часто путаются между двумя похожими библиотеками: PySide6 и PyQt6. Обе они предоставляют доступ к Qt6 из Python, и с точки зрения написания кода они почти неразличимы. Однако есть два важных отличия, о которых полезно знать:

- **Разработчик:** PySide6 создан и поддерживается самой компанией Qt (официальные биндинги), а PyQt6 — сторонней компанией Riverbank Computing.
- **Лицензия** — это самое существенное различие для практического применения:
 - PySide6 использует лицензию LGPL, что позволяет использовать библиотеку бесплатно в любых проектах, включая коммерческие и закрытые.
 - PyQt6 использует лицензию GPL, которая в большинстве случаев требует открытия исходного кода вашего приложения, либо покупки коммерческой лицензии.

Таким образом, PySide6 становится предпочтительным, а зачастую и единственным законным выбором для бизнеса, стартапов и индивидуальных разработчиков, создающих проприетарное программное обеспечение. При этом для open-source проектов под GPL и академических исследований обе библиотеки равноправны, и выбор может диктоваться личными предпочтениями или доступностью обучающих материалов (которых исторически больше для PyQt).

4.2. ШАГ 1 — СОЗДАНИЕ ГЛАВНОГО ОКНА

MainWindow наследуется от QMainWindow и выступает контроллером приложения, связывая сцену, панели управления и пользовательские действия.

4.2.1. Настройка окна

Метод `setupWindow` задаёт заголовок, размеры окна, и размечает меню приложения.

```
def setupWindow(self) -> None:
    self.setWindowTitle("ИИС Дорожной системы")
    self.resize(1100, 700)

    menu = self.menuBar().addMenu("Файл")

    save_action = menu.addAction("Сохранить")
    load_action = menu.addAction("Загрузить")

    events_action = self.menuBar().addAction("События")
    events_action.triggered.connect(self.openEventsWindow)

    save_action.triggered.connect(self.save)
    load_action.triggered.connect(self.load)
```

4.2.2. Компоновка интерфейса

setupLayout создаёт сцену (World) и помещает её в центральный виджет (WorldView). Слева добавляется панель управления (ControlPanel), справа — панель свойств (PropertiesPanel). Здесь также настраиваются сигналы: клик по клетке вызывает добавление сущности, выбор объекта обновляет панель свойств, а действия с панели управления управляют режимом светофоров и добавлением машин на сцену.

```
def setupLayout(self) -> None:
    self.world = World()
    self.world.onWorldCellClicked.connect(self.onWorldCellClicked)
    self.world.selectionChanged.connect(self.onSelectionChanged)
    self.setCentralWidget(WorldView(self.world))

    self.controlPanel = ControlPanel(self)
    self.controlPanel.entityFactorySelected.connect(self.onFactorySelected)
    self.controlPanel.spawnCarsClicked.connect(self.onSpawnCarsClicked)
    self.controlPanel.tlModeChanged.connect(self.onTlModeChanged)
    self.addDockWidget(Qt.DockWidgetArea.LeftDockWidgetArea, self.controlPanel)

    self.propertiesPanel = PropertiesPanel()
    self.addDockWidget(Qt.DockWidgetArea.RightDockWidgetArea, self.propertiesPanel)
)
```

4.2.3. Сохранение и загрузка

Методы save и load открывают диалог выбора файла и вызывают утилиты saveWorld / loadWorld, которые сериализуют состояние сцены в JSON и восстанавливают его оттуда.

```

def save(self):
    path, _ = QFileDialog.getSaveFileName(self, "Сохранить", "", "JSON (*.json)")

    if path:
        if not path.endswith(".json"):
            path += ".json"
        saveWorld(self.world, path)

def load(self):
    path, _ = QFileDialog.getOpenFileName(self, "", "Загрузить", "JSON (*.json)")
    if path:
        loadWorld(self.world, path)

```

4.3. ШАГ 2 — КОНСТАНТЫ И УТИЛИТЫ

Перед реализацией сцены необходимо определить базовые константы и вспомогательные функции, которые используются всеми компонентами приложения.

4.3.1. Константы

SRC_PATH, BASE_PATH и MEDIA_PATH определяют структуру каталогов проекта с помощью объектно ориентированной, мультиплатформенной библиотеки работы с путями для работы на любой машине.

```

SRC_PATH = Path(__file__).parent
BASE_PATH = SRC_PATH.parent
MEDIA_PATH = BASE_PATH / "media"

```

CELL_SIZE задаёт размер клетки в пикселях (35×35), а GRID_DIM определяет размер сетки 50×50 клеток.

```

CELL_SIZE = 35
GRID_DIM = 50

```

Словарь DIR2ROT сопоставляет направления с углами поворота текстур (в градусах). DIR2OFFSET определяет смещения по координатной сетке: движение на север уменьшает Y, на юг — увеличивает, на запад уменьшает X, на восток — увеличивает. Список CLOCKWISE задаёт порядок направлений по часовой стрелке, что удобно для расчётов поворотов

```

DIR2ROT: dict[Direction, int] = {
    Direction.N: 0,
    Direction.S: 180,
    Direction.W: -90,
    Direction.E: 90,
}

DIR2OFFSET: dict[Direction, QPoint] = {
    Direction.N: QPoint(0, -1),
    Direction.S: QPoint(0, 1),
    Direction.W: QPoint(-1, 0),
    Direction.E: QPoint(1, 0),
}

CLOCKWISE = [Direction.N, Direction.E, Direction.S, Direction.W]

```

4.3.2. Утилиты

getMediaPath возвращает абсолютный путь к медиафайлу (изображению) по его имени. getPrettyTimestamp преобразует временную метку в человекочитаемый формат для логирования событий

```

def getMediaPath(media_name: str) -> str:
    return str(MEDIA_PATH / media_name)

def getPrettyTimestamp(timestamp: float) -> str:
    return datetime.fromtimestamp(timestamp).strftime("%Y-%m-%d %H:%M:%S")

```

Функции преобразования координат: scenePosToCell переводит координаты сцены в клетку с округлением вниз (используется для определения клетки под курсором мыши), snapScenePosToCell — с математическим округлением, cellToScenePos выполняет обратное преобразование из клетки в пиксельные координаты сцены

```

def scenePosToCell(pos: QPoint | QPointF) -> QPoint:
    return QPoint(
        math.floor(pos.x() / CELL_SIZE),
        math.floor(pos.y() / CELL_SIZE),
    )

def snapScenePosToCell(pos: QPoint | QPointF) -> QPoint:
    return QPoint(
        round(pos.x() / CELL_SIZE),
        round(pos.y() / CELL_SIZE),
    )

def cellToScenePos(cell: QPoint) -> QPoint:
    return QPoint(
        cell.x() * CELL_SIZE,
        cell.y() * CELL_SIZE,
    )

```

getDirectionFromOffset возвращает направление по вектору смещения. rotateDirection вычисляет новое направление после выполнения манёвра (поворот

налево, направо, разворот или движение прямо), используя порядок CLOCKWISE и словарь дельт

```
def getDirectionFromOffset(offset: QPoint) -> Direction:
    for direction, dir_offset in DIR2OFFSET.items():
        if offset == dir_offset:
            return direction
    return Direction.N

def rotateDirection(
    direction: Direction, turn: TurnType
) -> Direction:
    delta = {
        TurnType.FORWARD: 0,
        TurnType.RIGHT: 1,
        TurnType.UITURN: 2,
        TurnType.LEFT: -1,
    }
    return CLOCKWISE[(CLOCKWISE.index(direction) + delta[turn]) % 4]
```

5.4. ШАГ 3 — СЦЕНА СИМУЛЯЦИИ — КЛАСС WORLD

World наследуется от QGraphicsScene и выступает контейнером, хранящим все сущности сцены, а также предоставляет методы для их поиска, навигации и генерации маршрутов. При инициализации задаются цвет фона и сетки, устанавливается прямоугольник сцены с центром в нулевых координатах, а также создаётся и запускается таймер симуляции, который с интервалом tickMs управляет поведением всех сущностей.

5.4.1. Поиск и навигация по сцене

World предоставляет три основных метода для получения сущностей. entities возвращает все объекты указанного типа (или типов). get возвращает сущности в конкретной клетке. getNeighbors — сущности, находящиеся в соседних клетках согласно предопределённым направлениям (DIR2OFFSET)

```

def entities[T: BaseEntity] (
    self, types: type[T] | tuple[type[T]]
) -> list[T]:
    return [e for e in self.items() if isinstance(e, types)]

def get[T: BaseEntity] (
    self, at: QPoint, types: type[T] | tuple[type[T]] =
BaseEntity
) -> list[T]:
    return [e for e in self.entities(types) if e.cell == at]

def getNeighbors[T: BaseEntity] (
    self, at: QPoint, types: type[T] | tuple[type[T]] =
BaseEntity
) -> list[T]:
    neighbors = []
    for _, offset in DIR2OFFSET.items():
        neighbors.extend(self.get(at + offset, types))
    return neighbors

```

5.4.2. Обработка нажатия по сцене

При клике на сцене `mousePressEvent` преобразует координаты события в клетку с помощью `scenePosToCell` и испускает сигнал `onWorldCellClicked`, который обрабатывается в `MainWindow`.

```

def mousePressEvent (
    self, event: QGraphicsSceneMouseEvent
) -> None:
    cell = scenePosToCell(event.scenePos())
    self.onWorldCellClicked.emit(cell)
    return super().mousePressEvent(event)

```

5.4.3. Отрисовка сетки

`drawBackground` рисует сетку: сначала заливает фон, затем линиями размечает клетки с шагом `CELL_SIZE` в пределах видимой области.

```

def drawBackground(self, painter: QPainter, rect: QRectF)
-> None:
    painter.fillRect(rect, self.backgroundColor)
    painter.setPen(QPen(self.gridColor, 0.4))
    left, right = int(rect.left()), int(rect.right())
    top, bottom = int(rect.top()), int(rect.bottom())
    first_left = left - (left % CELL_SIZE)
    first_top = top - (top % CELL_SIZE)
    for x in range(first_left, right, CELL_SIZE):
        painter.drawLine(x, top, x, bottom)
    for y in range(first_top, bottom, CELL_SIZE):
        painter.drawLine(left, y, right, y)

```

5.4.4. Обновление сущностей по таймеру

Каждый тик таймера вызывает `onTick`, который проходит по всем сущностям сцены и для каждой вызывает метод `tick`, передавая ему прошедшее время в секундах.

```
def onTick(self) -> None:
    for entity in self.entities():
        entity.tick(self.simTimer.interval() / 1000)
```

5.4.5. Генерация случайного маршрута

Метод `generateRandomPath` строит маршрут от заданной стартовой дороги длиной не более `maxLength` клеток. Алгоритм последовательно движется вперёд, обрабатывая два типа дорог: прямые участки и перекрёстки.

Маршрут начинается со стартовой дороги. Переменная `current` хранит текущую позицию на траектории.

```
def generateRandomPath(
    self, start: StraightRoad, maxLength: int = 60
) -> list[Road]:
    path: list[Road] = [start]
    current = start
```

На каждой итерации алгоритм запрашивает у сцены дорогу впереди (`getCellOffset(1)` возвращает смещение на одну клетку вперёд относительно направления текущей дороги). Если впереди нет дороги, маршрут прерывается

```
for _ in range(maxLength):
    ahead = self.get(current.getCellOffset(1), Road)
    if not ahead:
        break
    ahead = ahead[0]
```

Если впереди прямая дорога, проверяется совпадение направлений: поворот на прямой дороге невозможен. При успешной проверке дорога добавляется в маршрут, и движение продолжается.

```
if isinstance(ahead, StraightRoad):
    if current.direction != ahead.direction:
        break
    path.append(ahead)
    current = ahead
    continue
```

При достижении перекрёстка алгоритм получает все возможные варианты проезда через функцию `expandCrossroadPaths`, которая возвращает последовательности смещений для каждого типа поворота (прямо, налево, направо, разворот). Варианты перемешиваются случайным образом. Для каждого варианта проверяется, существует ли на выходе прямая дорога и можно ли её проехать (`canBePassed`)

```

if isinstance(ahead, CrossRoad):
    expandedPaths = list(
        expandCrossroadPaths(current.direction).items()
    )
    random.shuffle(expandedPaths)

    for turnType, pathSteps in expandedPaths:
        dst = self.get(current.cell + pathSteps[-1], StraightRoad)
        if not dst or not dst[0].canBePassed():
            continue
        for stepOffset in pathSteps:
            road = self.get(current.cell + stepOffset, Road)
            if not road:
                break # if gap in path then abort turn
            path.append(road[0])
        current = dst[0]
    break

```

Если выходная дорога доступна, алгоритм последовательно добавляет в маршрут все клетки перекрёстка согласно предвычисленным смещениям. Текущей позицией становится выездная дорога, после чего цикл завершается (перекрёсток пройден). Метод возвращает построенный маршрут.

5.5. ШАГ 4 — БАЗОВЫЙ КЛАСС СУЩНОСТИ — BASEENTITY

`BaseEntity` наследуется от `QGraphicsPixmapItem` и служит фундаментом для всех объектов на сцене (дорог, машин, светофоров и т.д.). Он инкапсулирует клеточную позицию, автоматическое масштабирование текстур, обработку перемещения и валидацию размещения.

5.5.1. Инициализация

Конструктор принимает два параметра: `zIndex` (порядок отрисовки) и `movable` (возможность перемещать объект мышью). По Специальность 09.02.08 — Интеллектуальные Интегрированные Системы Методическое пособие 21 умолчанию все сущности являются выделяемыми, а если `movable` равен `True`, им также добавляется

флаг перемещения. Начальная клетка определяется преобразованием текущей позиции в координаты сетки с помощью `scenePosToCell`.

```
class BaseEntity(QGraphicsPixmapItem):
    def __init__(self, zIndex: int = 0, movable: bool = True) -> None:
        super().__init__()

        flags = QGraphicsItem.GraphicsItemFlag.ItemIsSelectable
        if movable:
            flags |= QGraphicsItem.GraphicsItemFlag.ItemIsMovable
        self.setFlags(flags)

        self.setZValue(zIndex)

        self.cell = scenePosToCell(self.pos())
```

4.5.2. Поведение по таймеру

Метод `tick` (Листинг 20) вызывается каждым таймером сцены (передается время в секундах, прошедшее с предыдущего вызова). В базовом классе он ничего не делает и предназначен для переопределения в дочерних классах, где реализуется специфическое поведение — движение машин, переключение светофоров и т.д.

```
def tick(self, dt: float) -> None:
    pass
```

5.5.3. Установка текстуры с автоматическим масштабированием

Метод `setPixmap` (Листинг 21) переопределяет поведение базового класса и автоматически масштабирует переданное изображение до размера клетки (`CELL_SIZE`). Масштабирование выполняется с сохранением пропорций (`KeepAspectRatio`) и использованием сглаживания (`SmoothTransformation`) для качественного отображения.

```
def setPixmap(self, pixmap: QPixmap):
    super().setPixmap(
        pixmap.scaled(
            CELL_SIZE,
            CELL_SIZE,
            Qt.AspectRatioMode.KeepAspectRatio,
            Qt.TransformationMode.SmoothTransformation,
        )
    )
```

5.5.4. Обработка перемещения

При отпускании кнопки мыши после перетаскивания сущности на `mouseReleaseEvent` координаты курсора преобразуются в клетку с помощью `snapScenePosToCell`. Если новая клетка совпадает со старой или размещение в ней невалидно, сущность возвращается на прежнюю позицию. В противном случае выполняется перемещение.

```
def mouseReleaseEvent(self, event: QGraphicsSceneMouseEvent) -> None:
    super().mouseReleaseEvent(event)
    cell = snapScenePosToCell(self.pos())

    if cell == self.cell or not self.validatePlacement(cell, self.world):
        self.setCell(self.cell)
        return
    eventDb.log(
        f"Move {type(self).__name__} from ({self.cell.x()}, {self.cell.y()}) to ({cell.x()}, {cell.y()})"
    )

    self.setCell(cell)
```

5.5.5. Установка клеточной позиции

Метод `setCell` обновляет внутреннее представление клетки (`self.cell`) и устанавливает графическую позицию сущности на сцене, преобразуя координаты клетки в пиксельные через `cellToScenePos`.

```
def setCell(self, cell: QPoint) -> None:
    self.cell = cell

    self.setPos(cellToScenePos(cell))
```

5.5.6. Валидация размещения

Метод `validatePlacement` проверяет, можно ли разместить сущность в заданной клетке. По умолчанию запрещается размещение двух сущностей одного типа в одной клетке. Дочерние классы могут переопределять этот метод для более сложных правил.

```
def validatePlacement(self, cell: QPoint, world: "World") -> bool:
    existingItems = world.get(cell, types=type(self))
    return len(existingItems) == 0
```

5.6. ШАГ 5 — СЕРИАЛИЗАЦИЯ И ЗАГРУЗКА СЦЕНЫ – МОДУЛЬ PERSISTENCE

Модуль persistence отвечает за сохранение состояния сцены в JSON-файл и восстановление из него. Это позволяет пользователю сохранять созданные сцены дорожной сети и загружать их при следующем запуске.

5.6.1. Сериализаторы сущностей

Serializer — это датакласс, который связывает пару функций для конкретного типа сущности: serialize преобразует объект в словарь, а deserialize восстанавливает объект из словаря

```
@dataclass
class Serializer[T: BaseEntity]:
    serialize: Callable[[T], dict]
    deserialize: Callable[[dict[str, Any]], T]
```

Словарь SERIALIZERS сопоставляет каждый класс сущности с его сериализатором. Для дорог сохраняется направление, для знаков — тип, для пешеходных переходов — ориентация. Перекрёстки, светофоры и пешеходы не имеют дополнительных параметров, поэтому их сериализаторы работают с пустыми словарями.

```
SERIALIZERS: dict[type[BaseEntity], Serializer] = {
    StraightRoad: Serializer(
        lambda e: {"direction": e.direction.name},
        lambda d: StraightRoad(Direction[d.get("direction", "N")]),
    ),
    ...
}
```

CLASS_MAP используется для обратного преобразования: по строковому имени класса (например, "StraightRoad") получается соответствующий класс Python.

```
CLASS_MAP: dict[str, type[BaseEntity]] = {cls.__name__: cls for cls in SERIALIZERS}
```

5.6.2. Сериализация сущности

Функция serializeEntity принимает сущность и возвращает словарь с четырьмя полями:

- Тип сущности
- X клетки
- Y клетки

- Данные необходимые для инициализации - результат работы `serialize`

Если для типа нет сериализатора, выбрасывается исключение.

```
def serializeEntity(entity: BaseEntity) -> dict:
    serializer = SERIALIZERS.get(type(entity))
    if not serializer:
        raise ValueError(f"No serializer for {type(entity).__name__}")
    return {
        "type": type(entity).__name__,
        "x": entity.cell.x(),
        "y": entity.cell.y(),
        "data": serializer.serialize(entity),
    }
```

5.6.3. Десериализация сущности

`deserializeEntity` выполняет обратное преобразование: по полю `"type"` определяется класс, извлекается соответствующий сериализатор, создаётся объект без позиции (через `deserialize`), а затем позиция устанавливается отдельным вызовом `setCell`.

```
def deserializeEntity(entityData: dict[str, Any]) -> BaseEntity:
    cls = CLASS_MAP[entityData["type"]]
    serializer = SERIALIZERS.get(cls)
    if not serializer:
        raise ValueError(f"No deserializer for {cls.__name__}")
    obj = serializer.deserialize(entityData.get("data", {}))
    obj.setCell(QPoint(entityData["x"], entityData["y"]))
    return obj
```

5.6.4. Сохранение сцены

`saveWorld` собирает все сущности со сцены, сериализует каждую через `serializeEntity` и записывает JSON-объект с корневым полем `"entities"` в указанный файл.

```
def saveWorld(world: World, path: str) -> None:
    entities = [serializeEntity(entity) for entity in world.entities()]
    with open(path, "w", encoding="utf-8") as f:
        json.dump({"entities": entities}, f, indent=2)
```

5.6.5. Загрузка сцены

loadWorld открывает JSON-файл, очищает текущую сцену и добавляет каждую десериализованную сущность на сцену.

```
def loadWorld(world: World, path: str) -> None:
    with open(path, "r", encoding="utf-8") as f:
        worldData = json.load(f)

    world.clear()

    entities = worldData["entities"]

    for entity in entities:
        world.addItem(deserializeEntity(entity))
```

ЗАКЛЮЧЕНИЕ

Настоящее методическое пособие представляет собой завершённый учебно-методический комплекс, разработанный для формирования у студентов специальности 09.02.08 «Интеллектуальные Интегрированные Системы» профессиональных компетенций в области проектирования и разработки программных приложений с графическим интерфейсом.

В ходе изучения материала пособия обучающиеся последовательно осваивают:

1. Архитектурные принципы разработки графических приложений:

- паттерн разделения данных и представления (Model-View);
- организацию многослойной модели данных для визуальных редакторов;
- механизмы обработки событий и управления жизненным циклом виджетов;

- принципы инкапсуляции и модульности при проектировании классов.

2. Технологические аспекты работы с современными фреймворками:

- создание пользовательских интерфейсов на базе PyQt6, PyQt5 и PySide6 с использованием виджетов, компоновщиков и графических инструментов;
- реализация активного игрового цикла в Pygame для приложений с высокой частотой обновления кадров;
- сравнительный анализ архитектурных подходов: событийно-ориентированная модель Qt vs. циклическая модель Pygame.

3. Интеграцию с внешними системами и компонентами:

- организацию журналирования событий в реляционную базу данных SQLite с использованием параметрических запросов для защиты от SQL-инъекций;
- настройку двустороннего обмена данными с микроконтроллером Arduino через последовательный интерфейс (pyserial);
- синхронизацию состояния графического интерфейса, бизнес-логики и физического оборудования.

4. Практические навыки отладки и сопровождения кода:

- диагностику типовых ошибок работы с файловыми путями, графическими ресурсами и асинхронными операциями;
- применение принципов устойчивого программирования: обработка исключений, проверка валидности данных, управление ресурсами;

- документирование архитектурных решений и ведение журнала изменений.

Рекомендации для дальнейшего изучения:

- углублённое изучение паттернов проектирования (MVC, MVVM, Observer) для масштабирования приложений;
- освоение методов модульного тестирования графических интерфейсов (pytest-qt, unittest);
- изучение механизмов многопоточности в Qt (QThread, QThreadPool) для выполнения длительных операций без блокировки интерфейса;
- знакомство с кроссплатформенной сборкой приложений (PyInstaller, Nuitka) и распространением программного обеспечения.

Материал пособия полностью соответствует требованиям Федерального государственного образовательного стандарта среднего профессионального образования по специальности 09.02.08, рабочей программе дисциплины МДК.03 и может быть использован:

- для проведения практических занятий под руководством преподавателя;
- для организации самостоятельной работы студентов;
- в качестве методической основы для подготовки к демонстрационному экзамену;
- как справочный материал при выполнении курсового и выпускного квалификационного проектирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : национальный стандарт Российской Федерации : введен 2019-07-01. – Москва : Стандартинформ, 2018. – 142 с. – Текст : непосредственный.
2. Об образовании в Российской Федерации : Федеральный закон от 29.12.2012 № 273-ФЗ (ред. от 24.04.2025) // КонсультантПлюс : справ.-правовая система. – Москва, 2026. – URL: http://www.consultant.ru/document/cons_doc_LAW_140174/ (дата обращения: 08.04.2026). – Текст : электронный.
3. ФГОС СПО по специальности 09.02.08 «Интеллектуальные Интегрированные Системы» : приказ Минобрнауки России от 14.06.2022 № 510 // Министерство науки и высшего образования Российской Федерации : офиц. сайт. – Москва, 2022. – URL: <https://fgos.ru/fgos/fgos-09-02-08/> (дата обращения: 18.04.2026). – Текст : электронный.
4. Python Software Foundation. Python 3.10 Documentation : официальная документация [Электронный ресурс]. – 2021–2026. – URL: <https://docs.python.org/3.10/> (дата обращения: 18.04.2026). – Текст : электронный.
5. The Qt Company. Qt 6 Documentation : официальная документация фреймворка Qt [Электронный ресурс]. – 2021–2026. – URL: <https://doc.qt.io/qt-6/> (дата обращения: 18.04.2026). – Текст : электронный.
6. Riverbank Computing. PyQt6 Reference Guide : документация по Python-привязкам PyQt6 [Электронный ресурс]. – 2021–2026. – URL: <https://www.riverbankcomputing.com/static/Docs/PyQt6/> (дата обращения: 18.04.2026). – Текст : электронный.
7. The Qt Company. PySide6 Documentation : официальные биндинги Python для Qt6 [Электронный ресурс]. – 2021–2026. – URL: <https://doc.qt.io/qtforpython-6/> (дата обращения: 18.04.2026). – Текст : электронный.
8. Pygame Community. Pygame Documentation : документация библиотеки Pygame [Электронный ресурс]. – 2026. – URL: <https://www.pygame.org/docs/> (дата обращения: 18.04.2026). – Текст : электронный.

9. Arduino LLC. Arduino Reference : справочник по языку и функциям Arduino [Электронный ресурс]. – 2026. – URL: <https://www.arduino.cc/reference/> (дата обращения: 18.04.2026). – Текст : электронный.
10. SQLite Documentation. SQLite Library : документация встроенной СУБД SQLite [Электронный ресурс]. – 2026. – URL: <https://www.sqlite.org/docs.html> (дата обращения: 18.04.2026). – Текст : электронный.
11. PySerial Development Team. PySerial Documentation : документация библиотеки для работы с последовательными портами [Электронный ресурс]. – 2026. – URL: <https://pyserial.readthedocs.io/> (дата обращения: 18.04.2026). – Текст : электронный.
12. Летем, Дж. Программирование на Python : учебное пособие / Дж. Летем ; пер. с англ. – 3-е изд. – Москва : ДМК Пресс, 2023. – 512 с.
13. Саммерфилд, М. Программирование на PyQt5 : создание кроссплатформенных приложений / М. Саммерфилд ; пер. с англ. – Москва : ДМК Пресс, 2022. – 608 с.
14. Баллок, Дж. Изучаем Pygame : создание игр на языке программирования Python / Дж. Баллок ; пер. с англ. – Санкт-Петербург : Питер, 2021. – 464 с.
15. Microsoft. JSON (JavaScript Object Notation) : спецификация формата данных [Электронный ресурс]. – 2026. – URL: <https://www.json.org/json-ru.html> (дата обращения: 08.05.2026). – Текст : электронный.
16. Гайсина, Р. М. Архитектура программных систем : учебное пособие / Р. М. Гайсина. – Москва : ИНФРА-М, 2024. – 285 с. – (Высшее образование : Бакалавриат). – Текст : непосредственный.
17. Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос ; пер. с англ. – 4-е изд. – Санкт-Петербург : Питер, 2023. – 1120 с. – Текст : непосредственный.